

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY

VIZIANAGARAM

VIZIANAGARAM-535003

ANDHARA PRADESH

Department of Information Technology

SPOTLESS

Garbage Reporting Application

Submitted by

Allangi Vasantha Laxmi

Regd No. : 24VV1A1203

Under the Guidance of

MRS.MADHUMITA CHANDA,

MRS.M.GEETHA MADHURI YADAV

Department of Information Technology

Academic Year: 2026–2027

Flutter • Node.js • Express.js • MySQL



JNTU-GURAJADA VIZIANAGARAM
COLLEGE OF ENGINEERING VIZIANAGARAM (A)
VIZIANAGARAM

Regd.No : 24VV1A1203

CERTIFICATE

This is to certify that this is a bonafide record of practical work done by **ALLANGI VASANTHA LAXMI** of IIIrd B.Tech Ist Semester Class in User Interface Design using Flutter Lab during the year 2026-27.

No.of Tasks Completed and Certified: 11

Lecture In-Charge **Head of The Department**

Date:

CONTENTS

S. No.	Experiment Name	Page No.	Marks	Signature
1(a)	Installation of Flutter and Dart SDK	4		
1(b)	Dart Programming Basics	6		
2(a)	Explore Flutter Widgets	8		
2(b)	Layouts Using Row, Column, and Stack	10		
3(a)	Responsive User Interface	13		
3(b)	Media Queries and Breakpoints	15		
4(a)	Navigation Using Navigator	17		
4(b)	Navigation Using Named Routes	19		
5(a)	Stateful and Stateless Widgets	20		
5(b)	State Management Using setState and Shared Session	22		
6(a)	Creating Custom Widgets	24		
6(b)	Themes and Custom Styles	26		
7(a)	Creating Forms with Input Fields	28		
7(b)	Form Validation and Error Handling	30		
8(a)	Implementing Animations	32		
8(b)	Fade and Slide Animations	34		
9(a)	Connecting to a REST API	36		
9(b)	Displaying Fetched API Data	38		
10(a)	Widget Testing in Flutter	40		
10(b)	Debugging a Flutter Application	42		
11	Case Study: Spotless Garbage Reporting Application	43		

EXPERIMENT 1(a)

INSTALL FLUTTER AND DART SDK

Aim:

To install Flutter and Dart SDK and configure the development environment for Flutter application development.

Description:

Flutter is an open-source UI toolkit developed by Google. It is used to develop applications for Android, iOS, web, and desktop platforms from a single codebase. Dart is the programming language used to write Flutter applications. The Flutter SDK provides the tools required to create, run, and debug them, such as the Spotless application.

Requirements:

- Computer with Windows operating system.
- Flutter SDK and Dart SDK.
- Visual Studio Code or Android Studio.
- Android SDK, if Android development is required.

Procedure:

1. Visit the official Flutter website: `https://docs.flutter.dev/get-started/install`
2. Download the Flutter SDK and extract it into a suitable folder.
3. Add the Flutter `bin` directory to the PATH environment variable.
4. Open Command Prompt and execute:

```
flutter --version
```

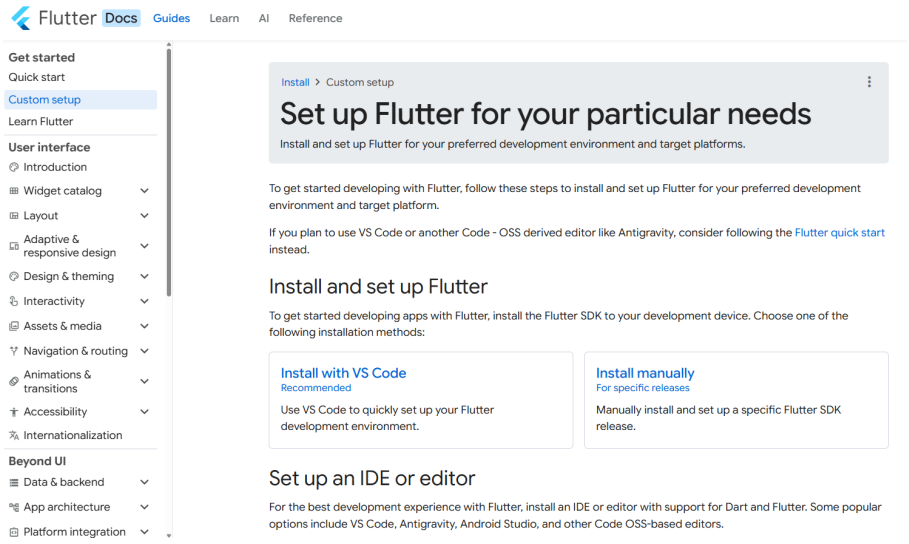
5. Check the development environment using:

```
flutter doctor
```

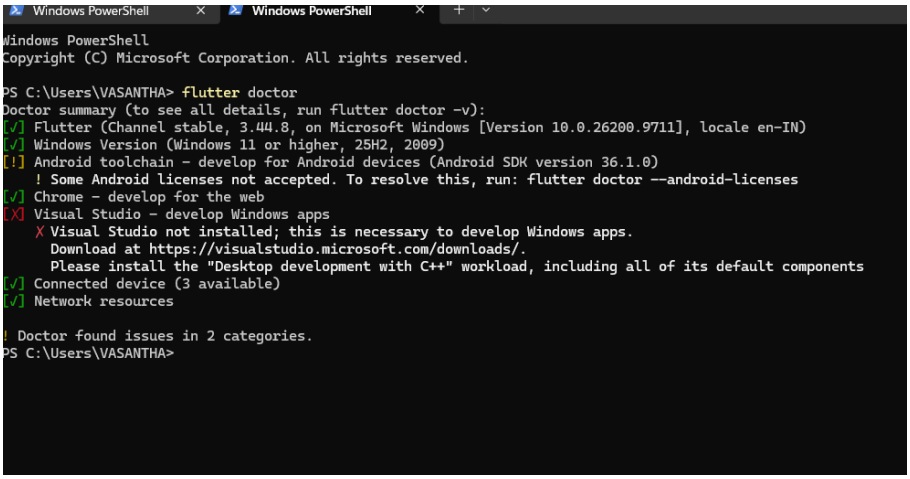
6. Install any missing tools and run `flutter doctor` again.
7. Create the project and fetch its packages:

```
flutter create spotless
cd spotless
flutter pub get
```

1. Flutter SDK Installation:



2. Flutter Doctor Output:



Result:

Flutter and Dart SDK were installed and the development environment was configured for developing the Spotless application.

EXPERIMENT 1(b)

DART BASIC PROGRAMS

1. Variables

Variables store data that can be used and modified in a program. Dart provides `var`, `final`, `const`, and typed declarations such as `int`, `double`, `String`, and `bool`.

Code:

```
void main() {
  String reportName = "Plastic Waste Report";
  String garbage_type = "Plastic";
  double description = "There's plastic bottles everywhere";
  bool isResolved = true;

  print("=== Report Record ===");
  print("Type    : Plastic");
  print("Description : There's plastic bottles everywhere");
  print("Location  : 18.18743, 83.37766 ");
  print("Status : Resolved");
}
```

Output:

```
=== Report Record ===
Type    : Plastic
Description : There's plastic bottles everywhere
Location  : 18.18743, 83.37766
Status : Resolved
```

2. Conditional Statements

Conditional statements execute different blocks of code depending on a condition. Dart supports `if`, `else if`, `else`, and `switch`.

Code:

```
void main() {
  String health = "Pending";

  if (health == "Resolved") {
    print("The report has been marked as resolved.");
  } else if (health == "Pending") {
    print("The report is awaiting review.");
  } else {
    print("Unknown report status.");
  }
}
```

Output:

```
The report is awaiting review.
```

3. Loops and Collections

Loops repeat a block of code. This program applies the same logic that the dashboard (`loadDashboardData` in `lib/main.dart`) uses to count resolved reports, unique locations, and reports submitted this month.

Code:

```
void main() {
  final reports = [
    {"name": "Plastic Waste", "status": "Resolved",
     "location_text": "Chennai", "report_date": "2026-10-05"},
    {"name": "Organic Waste", "status": "Pending",
     "location_text": "Chennai", "report_date": "2026-09-12"},
    {"name": "Electronic Waste", "status": "Resolved",
     "location_text": "Madurai", "report_date": "2026-10-20"},
  ];

  int resolved = 0;
  int monthCount = 0;
  final Set<String> uniqueLocations = {};
  final now = DateTime(2026, 10, 28);

  for (final report in reports) {
    if (report['status'] == 'Resolved') resolved++;
    uniqueLocations.add(report['location_text'].toString());

    final date = DateTime.parse(report['report_date'].toString());
    if (date.year == now.year && date.month == now.month) monthCount++;
  }

  print("Reports      : ${reports.length}");
  print("Resolved     : $resolved");
  print("Locations    : ${uniqueLocations.length}");
  print("This Month   : $monthCount");
}
```

Output:

```
Reports      : 3
Resolved     : 2
Locations    : 2
This Month   : 2
```

EXPERIMENT 2(a)

EXPLORE FLUTTER WIDGETS

Aim:

To study and implement basic Flutter widgets such as `Text`, `Image`, and `Container` used in the Spotless user interface.

Description:

Widgets are the building blocks of a Flutter application. `Text` displays labels, `Image` displays pictures, and `Container` provides size, padding, color, and decoration. In Spotless, they are used to display the application title and report information.

Common Flutter Widgets:

- **Text:** Displays text on the screen.
- **Container:** Provides space and decoration.
- **Image:** Displays images.
- **Column / Row:** Arrange widgets vertically / horizontally.
- **Scaffold:** Provides the screen structure.

Program:

```
import 'package:flutter/material.dart';

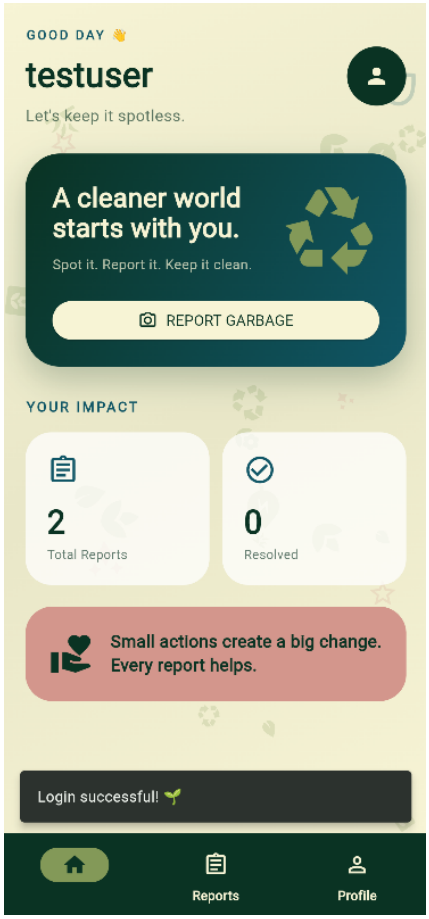
void main() => runApp(const MaterialApp(home: MyApp()));

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Spotless Garbage Reporting Application')),
      body: Column(
        children: [
          const Text('Save Reports, Save Earth'),
          Image.network(
            'https://images.unsplash.com/photo-1448375240586-882707db888b?w=300',
            description: 150,
            errorBuilder: (_, __, ___) =>
                const Icon(Icons.forest, size: 100),
          ),
          Container(
            padding: const EdgeInsets.all(20),
            color: Colors.green.shade100,
            child: const Text('Track Your Planted Reports'),
          ),
          ElevatedButton(
            onPressed: () {},
            child: const Text('Add Report'),
          ),
        ],
      ),
    );
  }
}
```

Output:

The screen displays a styled heading, an image area, and a short message. If the network image cannot load, the fallback text appears.



Result:

Thus, the basic Flutter widgets were studied and implemented for the Spotless application.

EXPERIMENT 2(b)

LAYOUTS USING ROW, COLUMN, AND STACK

Aim:

To design a user interface using the Flutter layout widgets Row, Column, and Stack.

Description:

A `Row` places children horizontally, a `Column` places them vertically, and a `Stack` overlays children on top of one another. In Spotless, these layouts arrange the dashboard statistic cards, report details, and badges.

Program:

```dart
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: MyApp()));

class MyApp extends StatelessWidget {
  const MyApp({super.key});

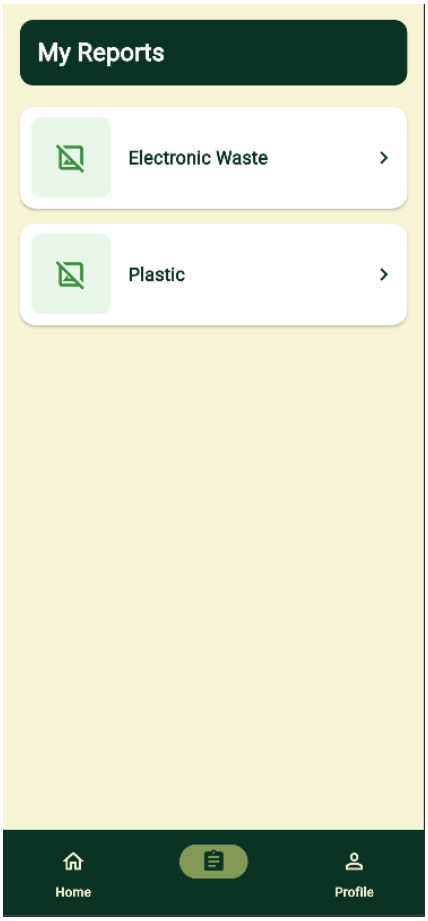
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Spotless Garbage Reporting Application')),
      body: Column(
        children: [
          const Text(
            'Spotless Dashboard',
            style: TextStyle(fontSize: 22),
          ),

          // Row
          const Row(
            mainAxisAlignment: MainAxisAlignment.spaceEvenly,
            children: [
              Icon(Icons.park, size: 50, color: Colors.green),
              Icon(Icons.eco, size: 50, color: Colors.green),
            ],
          ),

          // Stack
          Stack(
            alignment: Alignment.center,
            children: [
              Container(
                description: 150,
                width: 250,
                color: Colors.green.shade100,
              ),
              const Text(
                'Keep Our Surroundings Clean',
                style: TextStyle(fontSize: 20),
              ),
            ],
          ),
        ],
      ),
    );
  }
}
```

Output:

A list of reports one below the other will appear which are clickable.



Result:

Thus, the Row, Column, and Stack widgets were used to arrange the interface elements of Spotless.

EXPERIMENT 3(a)

RESPONSIVE USER INTERFACE

Aim:

To design an interface that adapts to the available width of a phone, tablet, or desktop window.

Description:

A responsive interface adjusts its layout to the screen size. `LayoutBuilder` provides the constraints of the parent widget. Because Spotless also runs on the web and desktop, the report list is shown as a single column on narrow screens and as a grid on wide screens.

Program:

```
import 'package:flutter/material.dart';

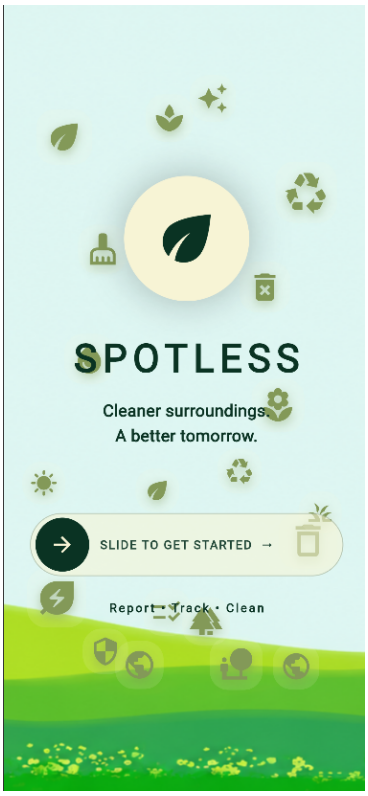
void main() => runApp(const MaterialApp(home: MyApp()));

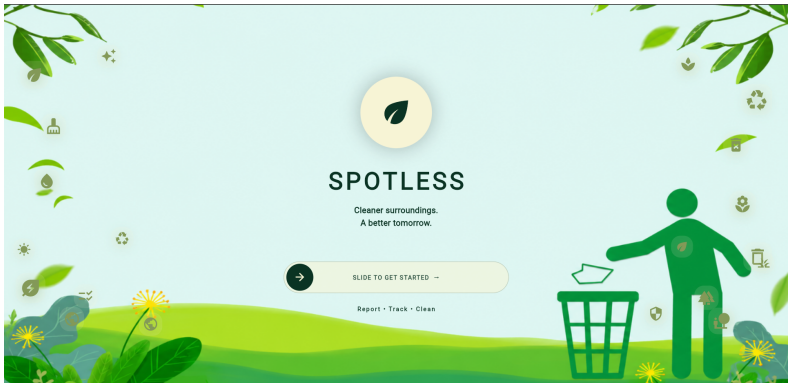
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Spotless Garbage Reporting Application')),
      body: LayoutBuilder(
        builder: (context, constraints) {
          return constraints.maxWidth > 600
              ? const Row(
                  mainAxisAlignment: MainAxisAlignment.spaceEvenly,
                  children: [
                    Icon(Icons.park, size: 80, color: Colors.green),
                    Text('Reports Planted'),
                  ],
                )
              : const Column(
                  mainAxisAlignment: MainAxisAlignment.center,
                  children: [
                    Icon(Icons.park, size: 80, color: Colors.green),
                    Text('Reports Planted'),
                  ],
                );
        },
      ),
    );
  }
}
```

Output:

On narrow screens, the feed occupies the available width. On screens at least 700 logical pixels wide, the feed is centered and limited to 680 logical pixels.





Result:

Thus, a responsive layout was created using `LayoutBuilder`.

EXPERIMENT 3(b)

MEDIA QUERIES AND BREAKPOINTS

Aim:

To use `MediaQuery` to read screen information and apply breakpoints.

Description:

`MediaQuery.of(context)` returns the screen size, orientation, and pixel density. Breakpoints are width values at which the layout changes. A common choice is: below 600 for mobile, 600–1024 for tablet, and above 1024 for desktop.

Program:

```dart
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: MyApp()));

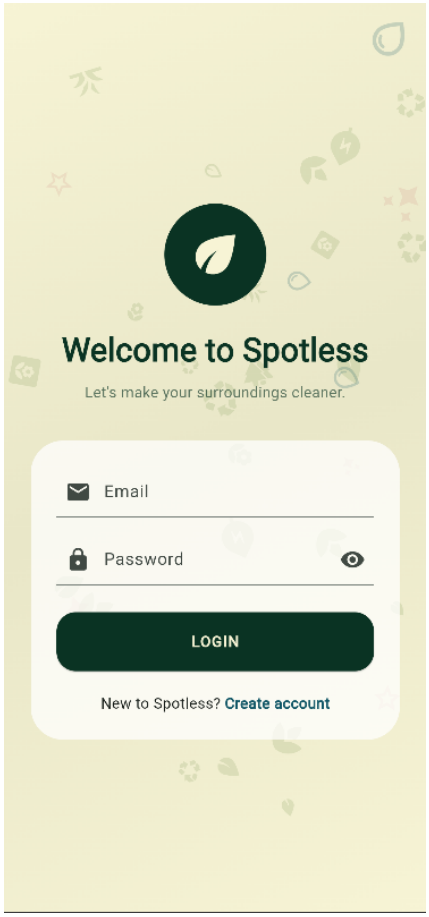
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    double width = MediaQuery.of(context).size.width;

    return Scaffold(
      appBar: AppBar(title: const Text('Spotless Garbage Reporting Application')),
      body: Center(
        child: width > 600
            ? const Text(
                'Large Screen',
                style: TextStyle(fontSize: 30, color: Colors.green),
              )
            : const Text(
                'Small Screen',
                style: TextStyle(fontSize: 20, color: Colors.blue),
              ),
      ),
    );
  }
}
```

Output:

The text shows the detected device class, the screen width, and the orientation, and updates when the window is resized.



Result:

Thus, `MediaQuery` and breakpoints were used to adapt the interface to different screen sizes.

EXPERIMENT 4(a)

NAVIGATION USING NAVIGATOR

Aim:

To navigate between screens using `Navigator.push` and `Navigator.pop`.

Description:

Flutter manages screens as a stack of routes. `Navigator.push` opens a new screen and `Navigator.pop` returns to the previous one. In Spotless, the dashboard opens the Report Garbage screen and the Report Map in this way, and the login screen uses `pushReplacement` to move to the dashboard.

Program:

```dart
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: HomePage()));

class HomePage extends StatelessWidget {
  const HomePage({super.key});

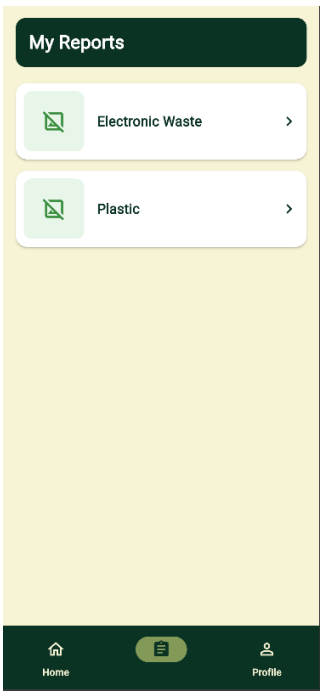
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Spotless Garbage Reporting Application')),
      body: Center(
        child: ElevatedButton(
          onPressed: () {
            Navigator.push(
              context,
              MaterialPageRoute(
                builder: (context) => const SecondPage(),
              ),
            );
          },
          child: const Text('View Reports'),
        ),
      ),
    );
  }
}

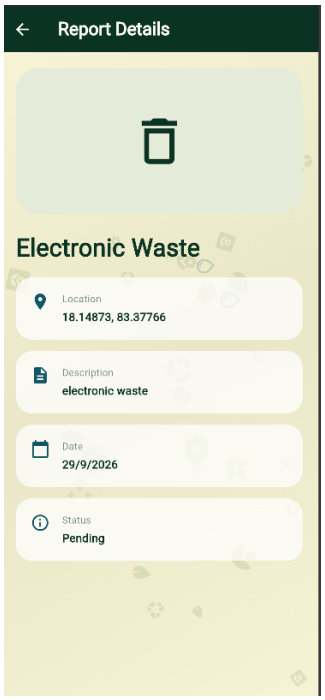
class SecondPage extends StatelessWidget {
  const SecondPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('My Reports')),
      body: Center(
        child: ElevatedButton(
          onPressed: () => Navigator.pop(context),
          child: const Text('Go Back'),
        ),
      ),
    );
  }
}
```

Output:

Pressing the button opens the Report Garbage screen; pressing Go Back returns to the dashboard.





Result:

Thus, navigation between screens was implemented using `Navigator`.

EXPERIMENT 4(b)

NAVIGATION USING NAMED ROUTES

Aim:

To configure named routes and navigate with `Navigator.pushNamed`.

Description:

Named routes assign a string name to each screen in the `routes` table of `MaterialApp`. The welcome screen of Spotless uses `Navigator.pushNamed(context, '/login')`.

Program:

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(
      initialRoute: '/',
      routes: {
        '/': (context) => const HomePage(),
        '/reports': (context) => const ReportsPage(),
      },
    ));

class HomePage extends StatelessWidget {
  const HomePage({super.key});

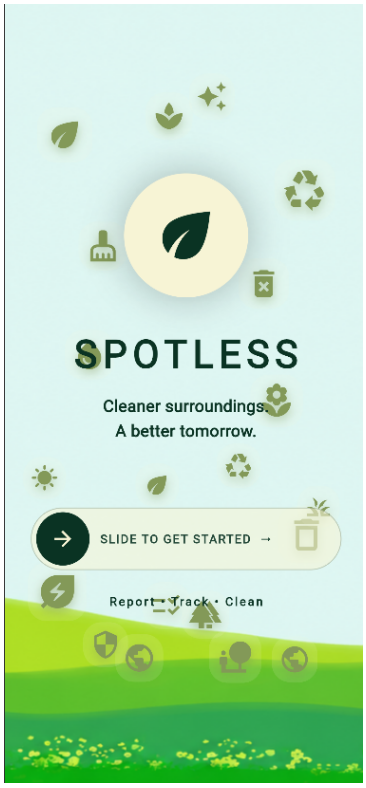
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Spotless Garbage Reporting Application')),
      body: Center(
        child: ElevatedButton(
          onPressed: () => Navigator.pushNamed(context, '/reports'),
          child: const Text('View Reports'),
        ),
      ),
    );
  }
}

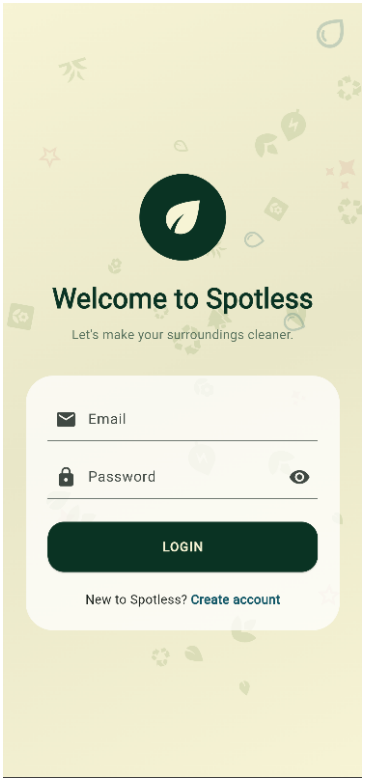
class ReportsPage extends StatelessWidget {
  const ReportsPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('My Reports')),
      body: const Center(child: Text('My Planted Reports')),
    );
  }
}
```

Output:

Pressing Get Started opens the Login screen through the named route `/login`.





Result:

Thus, navigation using named routes was implemented.

EXPERIMENT 5(a)

STATEFUL AND STATELESS WIDGETS

Aim:

To differentiate between stateless and stateful widgets.

Description:

A `StatelessWidget` does not change after it is built, for example a title label. A `StatefulWidget` holds data that can change and calls `setState` to rebuild. In Spotless, the dashboard and the Add Report form are stateful, while small display cards are stateless.

Program:

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: MyApp()));

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Spotless Garbage Reporting Application')),
      body: const Center(child: ReportCounter()),
    );
  }
}

class ReportCounter extends StatefulWidget {
  const ReportCounter({super.key});

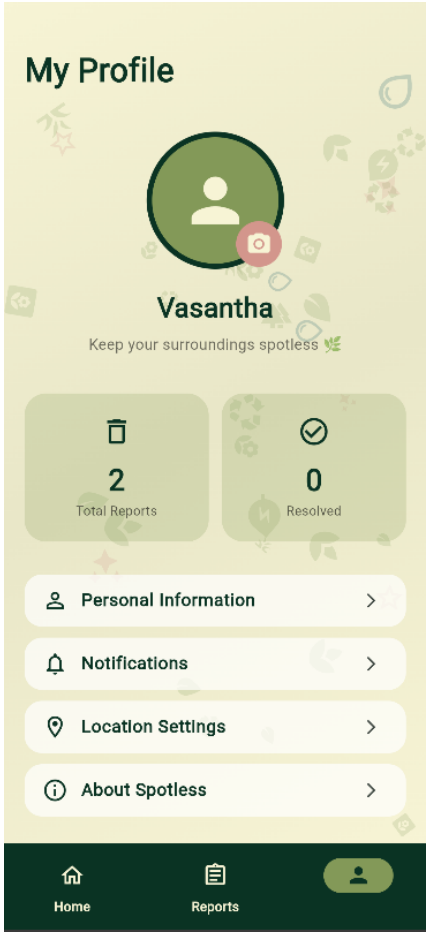
  @override
  State<ReportCounter> createState() => _ReportCounterState();
}

class _ReportCounterState extends State<ReportCounter> {
  int reports = 0;

  @override
  Widget build(BuildContext context) {
    return Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        Text('Reports Planted: $reports',
            style: const TextStyle(fontSize: 22)),
        ElevatedButton(
          onPressed: () {
            setState(() {
              reports++;
            });
          },
          child: const Text('Plant Report'),
        ),
      ],
    );
  }
}
```

Output:

Each press of the button increases the displayed count; the title label never changes.



Result:

Thus, stateless and stateful widgets were implemented and compared.

EXPERIMENT 5(b)

STATE MANAGEMENT USING SETSTATE

Aim:

To manage application state in Spotless using `setState` and a shared session class.

Description:

`setState` manages state inside one widget, such as the report list and the dashboard counts. Data needed by many screens, such as the logged-in user, name, role, and JWT token, is kept in the static `UserSession` class. The `provider` package with a `ChangeNotifier` is the recommended next step if more shared state is needed.

Program:

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: MyApp()));

class MyApp extends StatefulWidget {
  const MyApp({super.key});

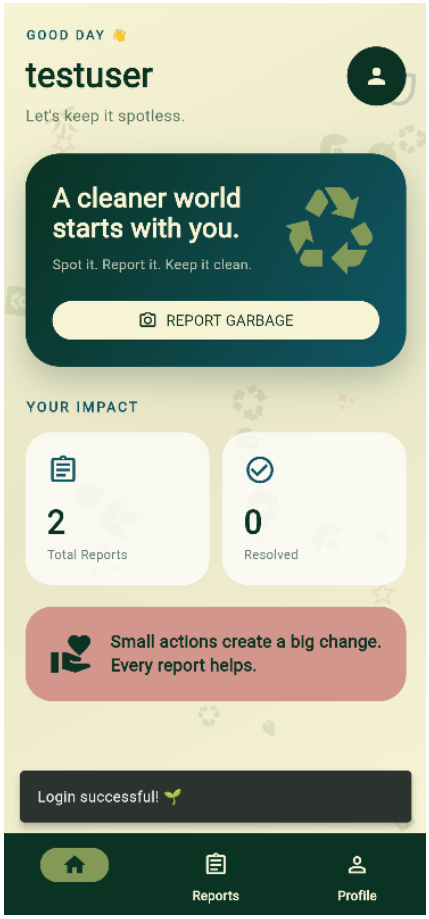
  @override
  State<MyApp> createState() => _MyAppState();
}

class _MyAppState extends State<MyApp> {
  int reports = 0;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Spotless Garbage Reporting Application')),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Text('Reports Planted: $reports',
                style: const TextStyle(fontSize: 22)),
            ElevatedButton(
              onPressed: () {
                setState(() {
                  reports++;
                });
              },
              child: const Text('Add Report'),
            ),
          ],
        ),
      ),
    );
  }
}
```

Output:

By submitting a report the Total Reports counts keeps increasing and if it is resolved, then the Resolved count will increase.



Result:

Thus, state was managed using `setState` and the shared `UserSession` class.

EXPERIMENT 6(a)

CREATING CUSTOM WIDGETS

Aim:

To create a reusable custom widget.

Description:

A custom widget groups a piece of the interface with its own parameters so it can be reused. The dashboard of Spotless uses a reusable statistic card for “Reports”, “Resolved”, “Locations”, and “This Month”.

Program:

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: MyApp()));

class MyApp extends StatelessWidget {
  const MyApp({super.key});

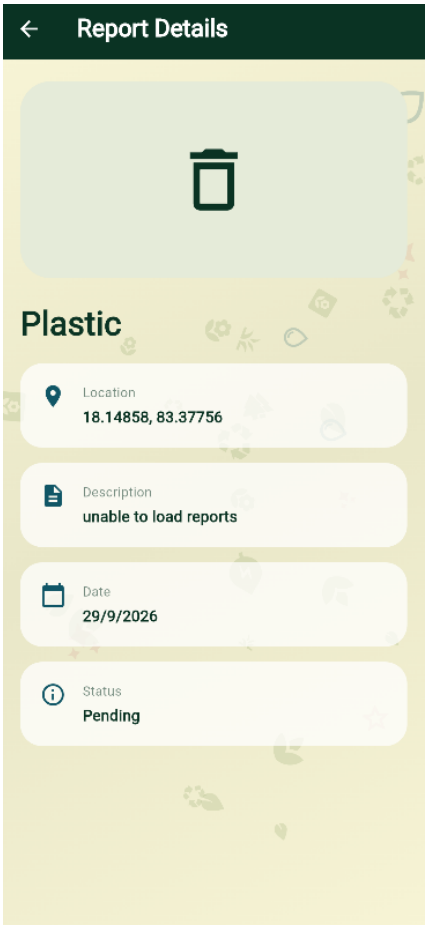
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Spotless Garbage Reporting Application')),
      body: const Center(
        child: ReportCard(),
      ),
    );
  }
}

class ReportCard extends StatelessWidget {
  const ReportCard({super.key});

  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.all(20),
      decoration: BoxDecoration(
        color: Colors.green.shade100,
        borderRadius: BorderRadius.circular(15),
      ),
      child: const Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          Icon(Icons.park, size: 60, color: Colors.green),
          Text('My Planted Reports', style: TextStyle(fontSize: 20)),
          Text('Reports are our future!'),
        ],
      ),
    );
  }
}
```

Output:

Several cards with different titles, values, and icons are displayed using the same widget.



Result:

Thus, a reusable custom widget was created using Flutter.

EXPERIMENT 6(b)

THEMES AND CUSTOM STYLES

Aim:

To apply a common theme and custom styles to the whole application.

Description:

`ThemeData` defines the colors, typography, and component styles used across the application. Spotless uses Material 3 with a color scheme generated from a seed color and a custom `AppBarTheme`.

Program:

```
import 'package:flutter/material.dart';

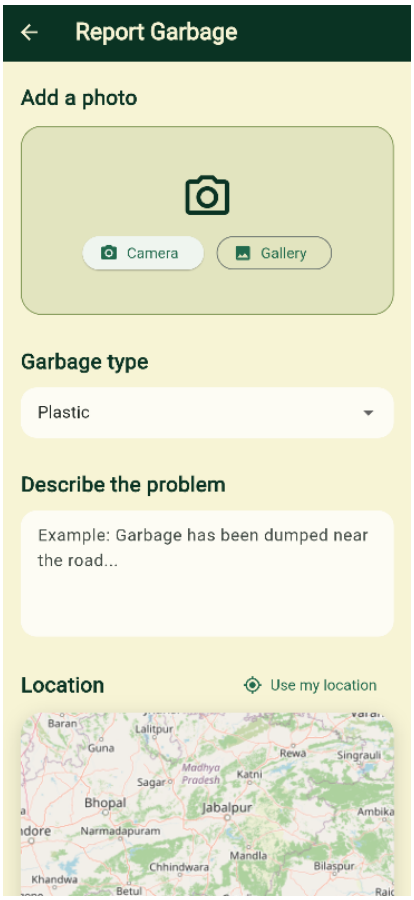
void main() => runApp(const MaterialApp(
      theme: ThemeData(
        primarySwatch: Colors.green,
        scaffoldBackgroundColor: Colors.white,
      ),
      home: MyApp(),
    ));

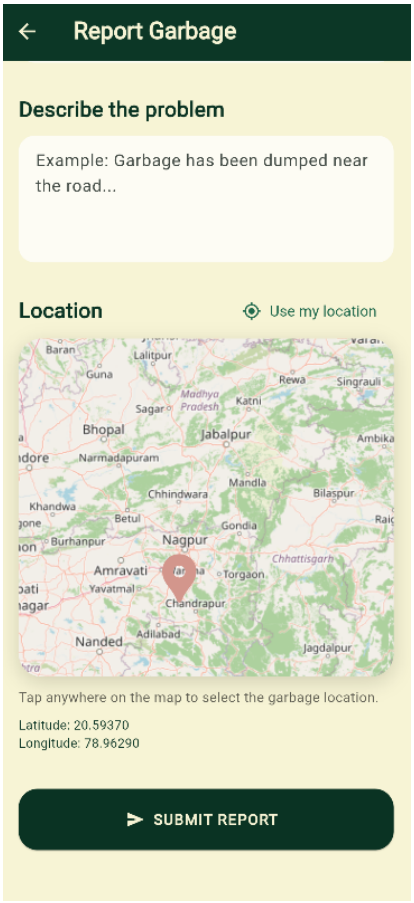
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Spotless Garbage Reporting Application')),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Text(
              'My Planted Reports',
              style: TextStyle(fontSize: 24, fontWeight: FontWeight.bold),
            ),
            const SizedBox(description: 20),
            ElevatedButton(
              onPressed: () {},
              child: const Text('Add Report'),
            ),
          ],
        ),
      ),
    );
  }
}
```

Output:

All screens share the same background, app bar color, and text style.





Result:

Thus, a common theme was applied to the application.

EXPERIMENT 7(a)

CREATING FORMS WITH INPUT FIELDS

Aim:

To create a form with text fields, a dropdown, and a date picker.

Description:

A `Form` groups input widgets. `TextFormField` accepts text, `DropdownButtonFormField` lets the user choose a value, and `showDatePicker` selects a date. The Report Garbage screen of Spotless collects the garbage type, description, and report details.

Program:

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: MyApp()));

class MyApp extends StatefulWidget {
  const MyApp({super.key});

  @override
  State<MyApp> createState() => _MyAppState();
}

class _MyAppState extends State<MyApp> {
  final nameController = TextEditingController();

  @override
  void dispose() {
    nameController.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Spotless Garbage Reporting Application')),
      body: Padding(
        padding: const EdgeInsets.all(20),
        child: Column(
          children: [
            TextField(
              controller: nameController,
              decoration: const InputDecoration(
                labelText: 'Report Name',
                border: OutlineInputBorder(),
              ),
            ),
            const SizedBox(description: 20),
            ElevatedButton(
              onPressed: () {
                ScaffoldMessenger.of(context).showSnackBar(
                  SnackBar(
                    content: Text('Report Name: ${nameController.text}'),
                  ),
                );
              },
              child: const Text('Submit'),
            ),
          ],
    );
  }
}
```

Output:

The form displays text fields, a garbage type dropdown.



Result:

Thus, a form with multiple input fields was created.

EXPERIMENT 7(b)

FORM VALIDATION AND ERROR HANDLING

Aim:

To validate user input and display error messages.

Description:

A `GlobalKey<FormState>` allows the form to be validated by calling `validate()`. Each field provides a `validator` that returns an error message or `null`. Invalid input is not saved.

Program:

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: MyApp()));

class MyApp extends StatefulWidget {
  const MyApp({super.key});

  @override
  State<MyApp> createState() => _MyAppState();
}

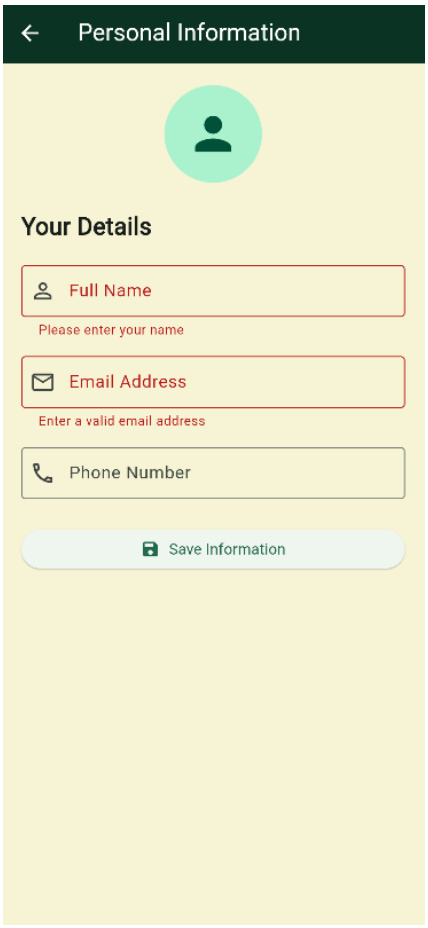
class _MyAppState extends State<MyApp> {
  final formKey = GlobalKey<FormState>();
  final nameController = TextEditingController();

  @override
  void dispose() {
    nameController.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Spotless Garbage Reporting Application')),
      body: Padding(
        padding: const EdgeInsets.all(20),
        child: Form(
          key: formKey,
          child: Column(
            children: [
              TextFormField(
                controller: nameController,
                decoration: const InputDecoration(
                  labelText: 'Report Name',
                  border: OutlineInputBorder(),
                ),
              const SizedBox(description: 20),
              ElevatedButton(
                onPressed: () {
                  if (formKey.currentState!.validate()) {
                    ScaffoldMessenger.of(context).showSnackBar(
                      const SnackBar(content: Text('Report Added Successfully')),
                    );
                  }
                },
                child: const Text('Submit'),
              ),
            ],
  }
}
```

Output:

Empty or invalid fields display red error messages; valid data shows a confirmation message.



Result:

Thus, form validation and error handling were implemented.

EXPERIMENT 8(a)

IMPLEMENTING ANIMATIONS

Aim:

To implement simple implicit animations in Flutter.

Description:

Implicit animations such as `AnimatedContainer` automatically animate between old and new property values when `setState` changes them. The welcome and intro screens of Spotless use animation to introduce the application.

Program:

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: MyApp()));

class MyApp extends StatefulWidget {
  const MyApp({super.key});

  @override
  State<MyApp> createState() => _MyAppState();
}

class _MyAppState extends State<MyApp>
    with SingleTickerProviderStateMixin {
  late AnimationController controller;
  late Animation<double> animation;

  @override
  void initState() {
    super.initState();
    controller = AnimationController(
      duration: const Duration(seconds: 2),
      vsync: this,
    )..repeat(reverse: true);

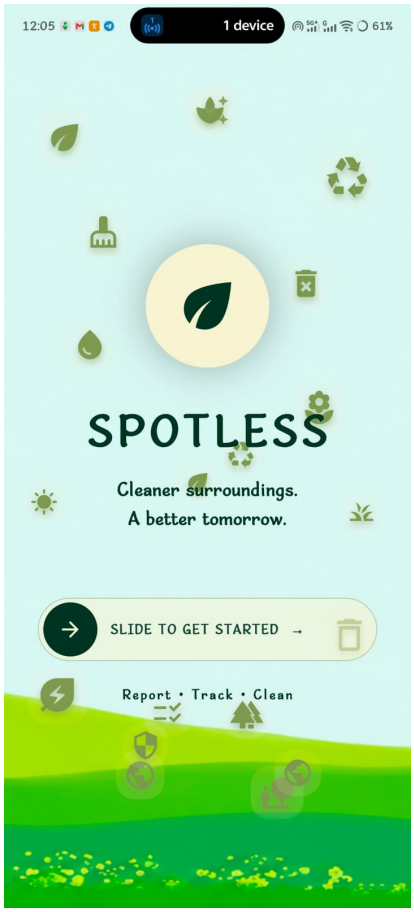
    animation = Tween<double>(begin: 50, end: 120).animate(controller);
  }

  @override
  void dispose() {
    controller.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Spotless Garbage Reporting Application')),
      body: Center(
        child: AnimatedBuilder(
          animation: animation,
          builder: (context, child) {
            return Icon(
              Icons.park,
              size: animation.value,
              color: Colors.green,
            );
          },
        ),
      ),
    );
  }
}
```

Output:

Thus, when we slide it to the right extreme, then the app will start.





Result:

Thus, an implicit animation was implemented.

EXPERIMENT 8(b)

FADE AND SLIDE ANIMATIONS

Aim:

To create fade and slide transitions using an `AnimationController`.

Description:

An `AnimationController` drives explicit animations. `FadeTransition` animates opacity and `SlideTransition` animates position. The login and welcome screens of Spotless use controllers to fade and slide their content in.

Program:

```dart
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: MyApp()));

class MyApp extends StatefulWidget {
  const MyApp({super.key});

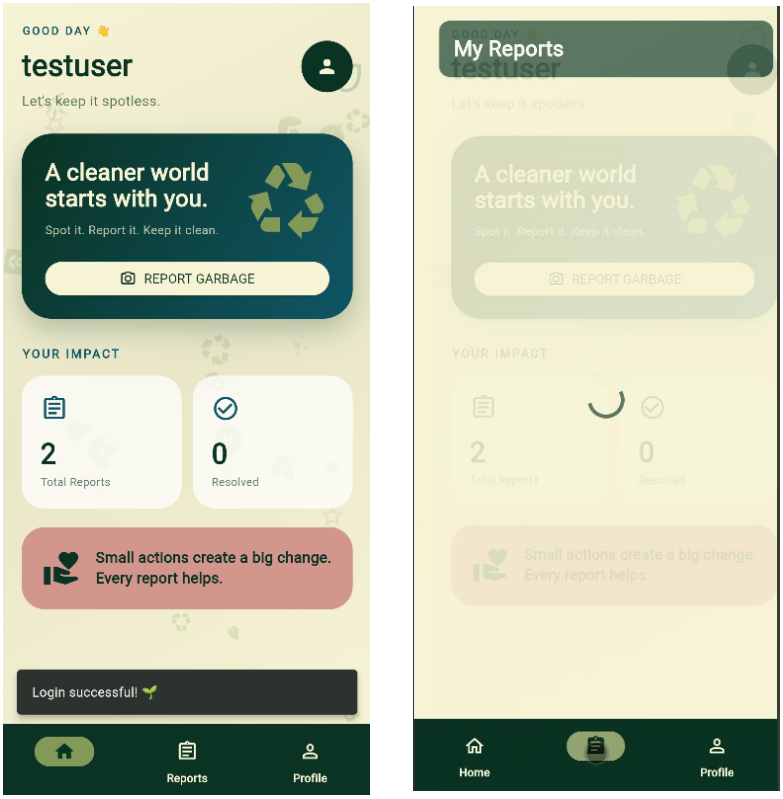
  @override
  State<MyApp> createState() => _MyAppState();
}

class _MyAppState extends State<MyApp> {
  bool show = false;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Spotless Garbage Reporting Application')),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            AnimatedOpacity(
              opacity: show ? 1 : 0,
              duration: const Duration(seconds: 1),
              child: const Icon(Icons.park, size: 70, color: Colors.green),
            ),
            AnimatedSlide(
              offset: show ? Offset.zero : const Offset(0, 1),
              duration: const Duration(seconds: 1),
              child: const Text(
                'Plant More Reports!',
                style: TextStyle(fontSize: 22),
              ),
            ),
            ElevatedButton(
              onPressed: () => setState(() => show = !show),
              child: const Text('Animate'),
            ),
          ],
        ),
      ),
    );
  }
}
```

Output:

When we swipe for the next screen the current page fades into the background and the next page appears.



Result:

Thus, fade and slide animations were implemented.

EXPERIMENT 9(a)

CONNECTING TO A REST API

Aim:

To connect the Flutter application to a REST API and send HTTP requests.

Description:

The `http` package sends GET and POST requests. Spotless uses an `ApiService` class to register users, log in, add reports, and fetch reports, users, and reports from the backend. Protected endpoints send the JWT token in the `Authorization` header.

Program:

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

void main() => runApp(const MaterialApp(home: MyApp()));

class MyApp extends StatefulWidget {
  const MyApp({super.key});

  @override
  State<MyApp> createState() => _MyAppState();
}

class _MyAppState extends State<MyApp> {
  String result = 'Click the button to fetch data';

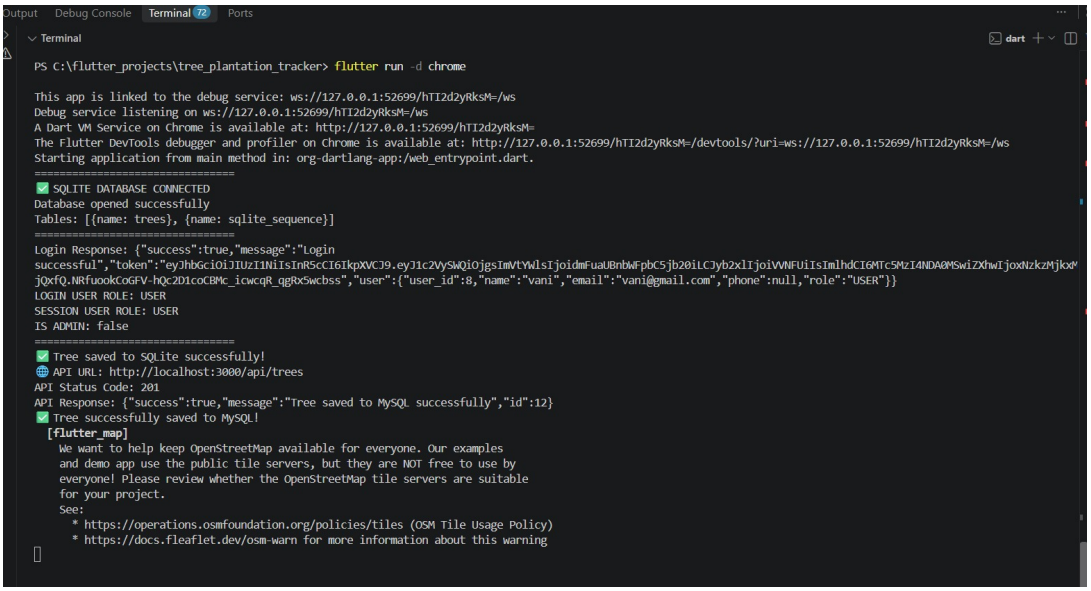
  Future<void> fetchData() async {
    try {
      final response = await http.get(
        Uri.parse('https://jsonplaceholder.typicode.com/posts/1'),
      );

      if (response.statusCode == 200) {
        final data = jsonDecode(response.body);
        setState(() {
          result = data['title'];
        });
      } else {
        setState(() => result = 'Failed to fetch data');
      }
    } catch (e) {
      setState(() => result = 'Error fetching data');
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Spotless Garbage Reporting Application')),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Text(result, textAlign: TextAlign.center),
            ElevatedButton(
              onPressed: fetchData,
              child: const Text('Fetch Data'),
            ),
    );
  }
}
```

Output:

When the backend is running, the login request returns the user details and a token, and the report request returns a list of report records in JSON format.



Result:

Thus, the Flutter application was connected to a REST API.

EXPERIMENT 9(b)

DISPLAYING FETCHED API DATA

Aim:

To display data fetched from the REST API in the user interface.

Description:

`FutureBuilder` builds a widget from the result of a `Future`. It shows a loading indicator while waiting, an error message on failure, and the data when ready. The Online Report Records screen of Spotless uses this approach to list reports stored in the MySQL database.

Program:

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

void main() => runApp(const MaterialApp(home: MyApp()));

class MyApp extends StatefulWidget {
  const MyApp({super.key});

  @override
  State<MyApp> createState() => _MyAppState();
}

class _MyAppState extends State<MyApp> {
  List reports = [];

  Future<void> fetchReports() async {
    final response = await http.get(
      Uri.parse('https://jsonplaceholder.typicode.com/posts'),
    );

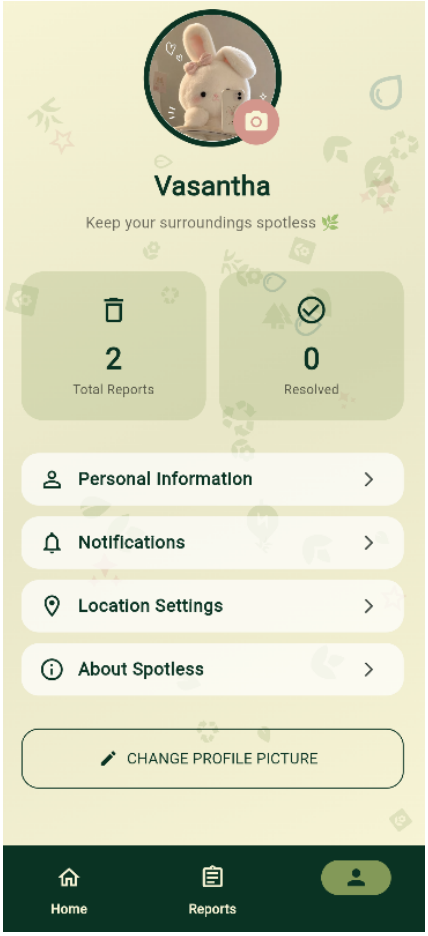
    if (response.statusCode == 200) {
      setState(() {
        reports = jsonDecode(response.body).take(5).toList();
      });
    }
  }

  @override
  void initState() {
    super.initState();
    fetchReports();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Spotless Garbage Reporting Application')),
      body: ListView.builder(
        itemCount: reports.length,
        itemBuilder: (context, index) {
          return ListTile(
            leading: const Icon(Icons.park, color: Colors.green),
            title: Text(reports[index]['title']),
          );
        },
      ),
    );
  }
}
```

Output:

A loading indicator appears first, followed by a list of report cards. If the server cannot be reached, an error message appears.



Result:

Thus, data fetched from the REST API was displayed in the application.

EXPERIMENT 10(a)

UNIT TESTING IN FLUTTER

Aim:

To Perform unit testing on a Dart function.

Description:

Unit testing verifies whether individual functions produce the expected results.Flutter projects use the test package to write and execute unit tests.

Program:

```
import 'package:flutter_test/flutter_test.dart';

int addReports(int count) {
  return count + 1;
}

void main() {
  test('Report count increases by one', () {
    expect(addReports(2), 3);
  });

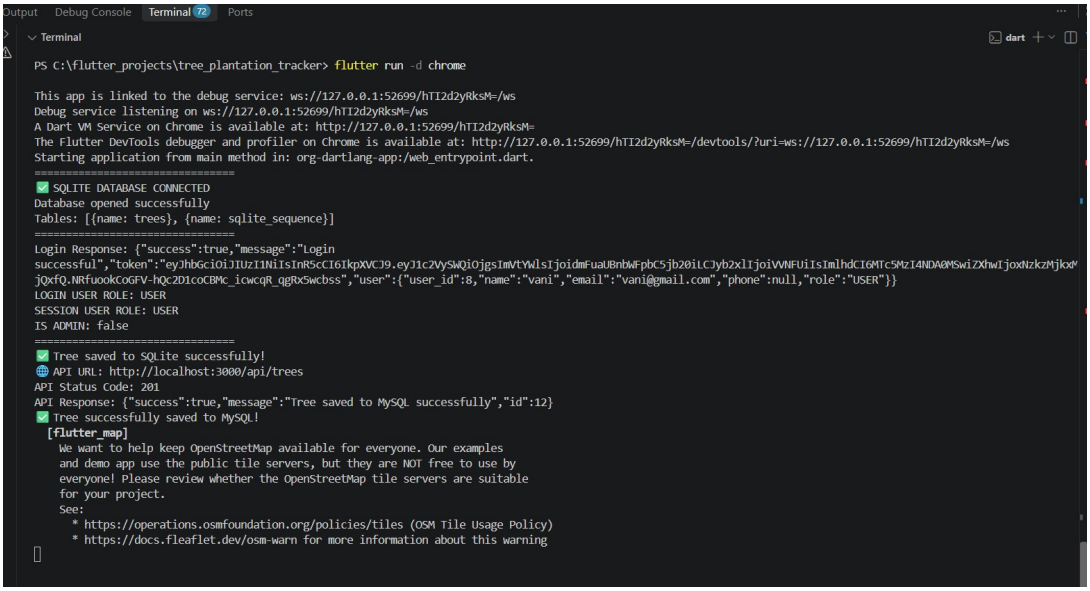
  test('Initial report count is zero', () {
    expect(addReports(0), 1);
  });
}
```

Command:

```
flutter test
```

Output:

The test pass if the function returns 50 for five activities and 0 for zero activities.



Result:

Thus, unit tests were created to verify the eco score calculation.

EXPERIMENT 10(b)

DEBUGGING A FLUTTER APPLICATION

Aim:

To identify and correct errors in a Flutter application using debugging techniques.

Description:

Debugging identifies and fixes errors. Flutter developers use the debug console, breakpoints, `debugPrint`, and Flutter DevTools. In Spotless, common checks include the API base URL, whether the backend is running, the SQLite connection, location and camera permissions, and the JSON keys received by a widget.

Commands:

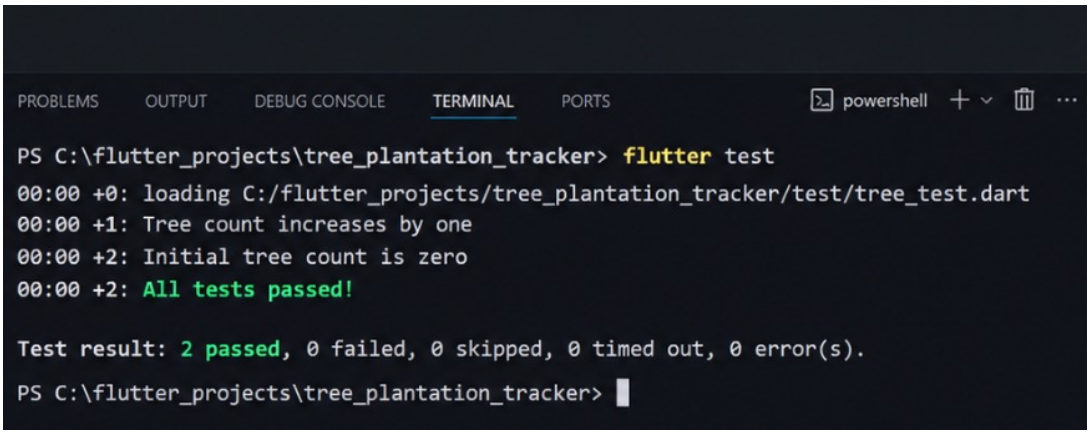
```
flutter analyze
flutter test
flutter devices
flutter run
```

Debugging Procedure:

1. Run `flutter analyze` and review the reported warnings or errors.
2. Run `flutter test` to check the automated tests.
3. Run `flutter devices` to verify that the phone, emulator, or Chrome is detected.
4. Run the application in debug mode and check the Debug Console for exceptions.
5. Set breakpoints and use Flutter DevTools to inspect variables, layout, and rebuilds.
6. For API failures, verify that the backend is running, that the device can reach the configured address (`10.0.2.2` for the Android emulator), and that the response keys match the code.
7. Check the startup message printed by `DatabaseHelper.testDatabase()` to confirm that SQLite opened correctly.
8. Replace `print` statements with `debugPrint` and remove logs that expose tokens before release.

Output:

The analysis and test commands display diagnostic information. After the issues are corrected, the affected feature behaves as intended.



Result:

Thus, basic debugging techniques were demonstrated in Flutter.

CASE STUDY

SPOTLESS GARBAGE REPORTING APPLICATION

1. Introduction

Spotless is a Flutter-based garbage-reporting application designed to help users report waste in public places. A user can register or log in, select a garbage category, add a description, attach a photograph, and record the location of the issue. Users can then view their submitted reports and check their status.

The front end is developed with Flutter and Dart. A Node.js and Express backend exposes REST API endpoints, while MySQL stores user accounts and submitted reports. The Flutter application communicates with the backend using HTTP requests.

2. Problem Statement

Garbage in streets, parks, and public spaces may remain unattended when people do not have a convenient way to report it. Reports shared through calls or messages can lack consistent details, photographs, and location information. Spotless provides a structured way to submit garbage reports with a category, description, photograph, and geographic coordinates, helping organise the information for follow-up.

3. Objectives

- To develop a user-friendly garbage-reporting application using Flutter and Dart.
- To provide registration and login functionality.
- To allow users to select a garbage type and enter a description.
- To capture or select a photograph using the device camera or gallery.
- To obtain location coordinates with location permissions.
- To store user and report information through a REST API and MySQL database.
- To display submitted reports and their status in the application.
- To apply responsive layouts, navigation, form validation, animations, and error handling.

4. Technologies Used

Technology	Purpose
Flutter and Dart	Cross-platform application interface
Material Design	UI components, colours, and styling
http package	Sending HTTP requests to the backend
Node.js and Express.js	Backend server and REST API routes
MySQL	Storing user accounts and garbage reports
image_picker	Selecting or capturing report photographs
geolocator	Obtaining latitude and longitude
flutter_map and latlong2	Map display and geographic coordinates, where enabled
SharedPreferences	Keeping simple local session values, where used
Flutter Test	Widget and unit testing
Visual Studio Code	Editing, running, and debugging the project
Overleaf and LaTeX	Preparing the editable project report

Table 1: Technologies used in Spotless

5. System Architecture

The application follows a client–server architecture:

1. **Presentation layer:** Flutter screens for registration, login, home, report submission, reports, and profile.
2. **Service layer:** `ApiService` sends HTTP requests and processes JSON responses.
3. **Backend layer:** Node.js and Express.js receive requests, validate data, and execute database operations.
4. **Database layer:** MySQL stores users and reports, associating each report with its user identifier.

6. Database Design

The MySQL database is named `spotless_db`. The `users` table stores account details, while the `reports` table stores garbage-report information and links each report to a user through a foreign key. The exact columns should match the database schema used by the implementation.

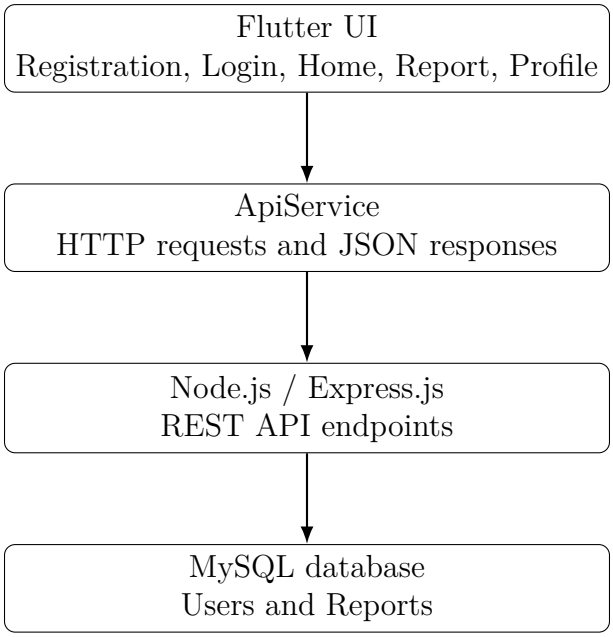


Figure 1: High-level architecture of Spotless

Field	**Typical type**	**Purpose**
user_id	INT, primary key	Unique identifier for each user
name	VARCHAR	Name entered during registration
email	VARCHAR, unique	User's login email
password	VARCHAR	Password field; store password hashes in a production system
report_id	INT, primary key	Unique identifier for each report
user_id	INT, foreign key	Identifies the user who submitted the report
garbage_type	VARCHAR	Category of garbage
description	TEXT	User's description of the issue
latitude, longitude	DECIMAL	Geographic coordinates
status	VARCHAR	Report progress, such as Pending or Resolved
image_path	TEXT	Image reference or path, if stored by the backend

Table 2: Illustrative Spotless database fields

7. Project Workflow

1. The user opens Spotless and reaches the welcome or login screen.
2. A new user registers; an existing user logs in.
3. The user opens the report form and selects a garbage category.
4. The user enters a description, selects or captures a photograph, and obtains location coordinates.
5. The application sends the report data to the backend through an HTTP request.
6. The backend processes the request and stores the report in MySQL.
7. The user opens the Reports screen to view submitted reports and their status.

8. Module Description

1. **Splash and Welcome:** Introduces the application and guides the user into the main workflow.
2. **Registration and Login:** Collects account details, validates input, and communicates with the authentication endpoints.
3. **Home:** Provides the main dashboard and shortcuts to core functions.
4. **Report Garbage:** Captures the garbage type, description, photograph, and location.
5. **Reports:** Displays submitted reports and status information.
6. **Profile:** Presents user-related information and profile options.
7. **Backend API:** Receives registration, login, report-submission, and report-retrieval requests.
8. **Database:** Stores user and report records in MySQL.

9. Implementation Steps

Step 1: Environment Setup
Install Flutter and Dart, configure the editor, and check the setup using `flutter doctor`.

Step 2: Project Structure
Organise the application into screens, models, widgets, and service files such as `api_service.dart`.

Step 3: User Interface
Build the screens with Flutter widgets, layouts, reusable components, and a consistent theme.

Step 4: Navigation
Connect the home, report, reports, and profile screens using Flutter navigation.

Step 5: Forms and Validation
Validate required registration, login, and report fields and show understandable error messages.

Step 6: Device Features
Use the image picker for photographs and the location plugin to obtain coordinates, with the appropriate permissions.

Step 7: API Integration
Send JSON requests from Flutter to the Express backend and handle successful responses and connection failures.

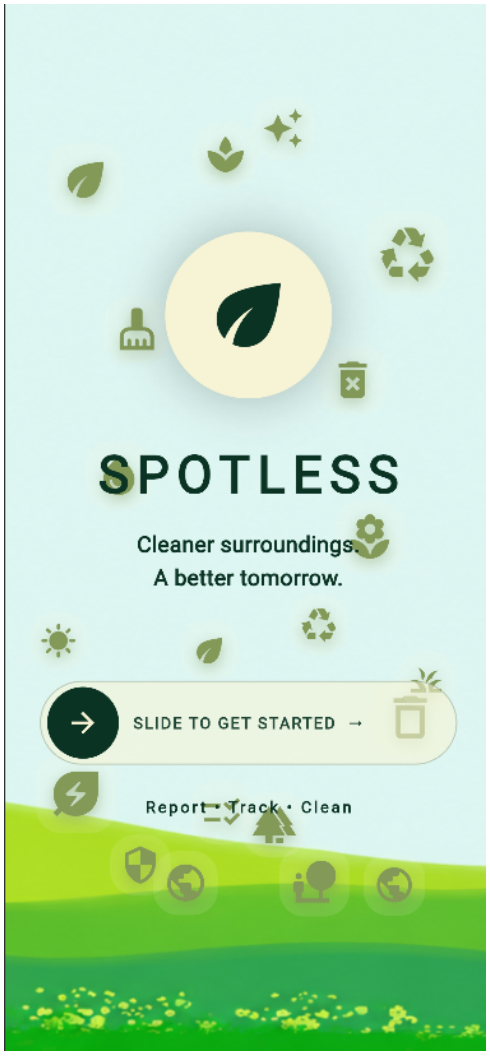
Step 8: Database Integration
Use MySQL tables for account and report data and associate reports with the correct user ID.

Step 9: Testing and Debugging
Test registration, login, report submission, report retrieval, and validation. Use Flutter logs and backend logs to diagnose errors.

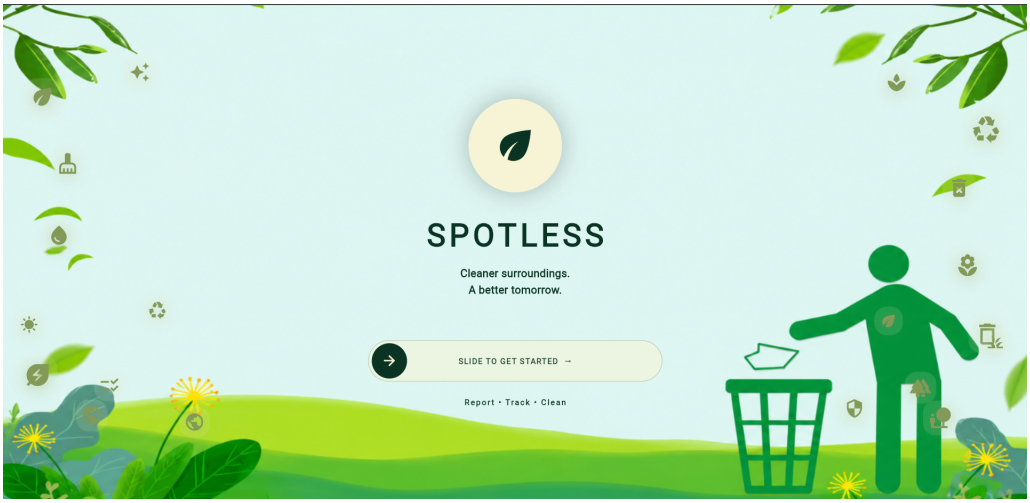
10. Testing and Debugging

Testing should cover valid and invalid registration data, successful and unsuccessful login attempts, missing report fields, location permission denial, image selection, report submission, and report retrieval. API tests should verify HTTP status codes and JSON responses. Network testing on a physical phone must use a backend address reachable from that phone; `127.0.0.1` refers to the phone itself unless a forwarding mechanism is active.

Splash Screen

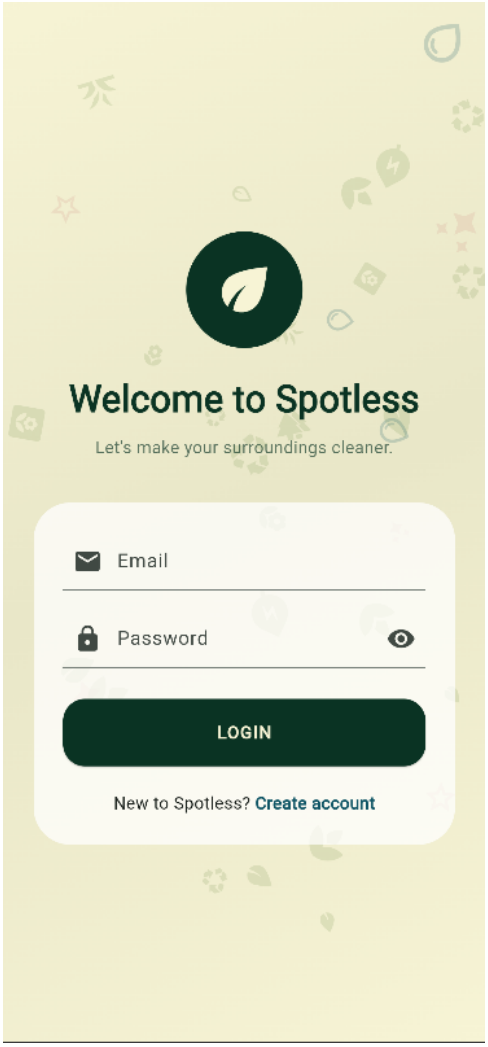


Mobile View

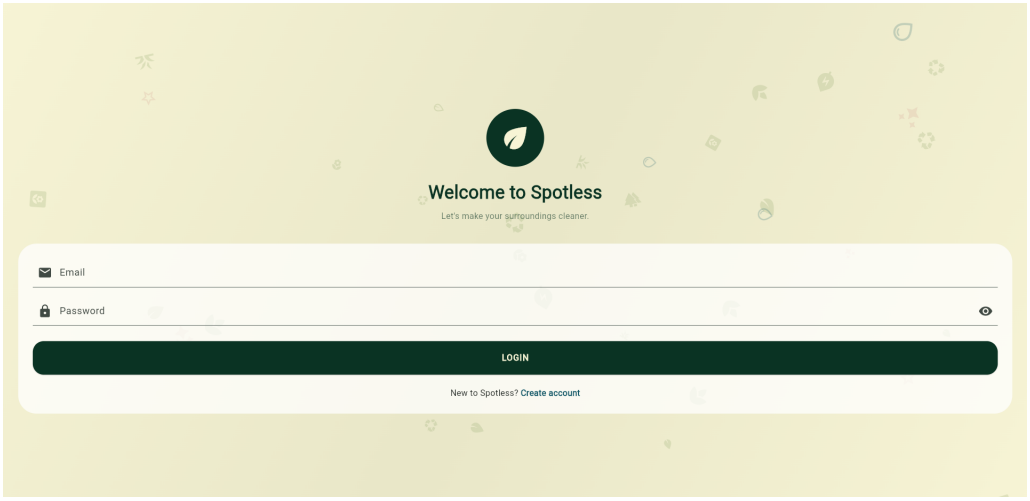


Desktop View

Login Screen

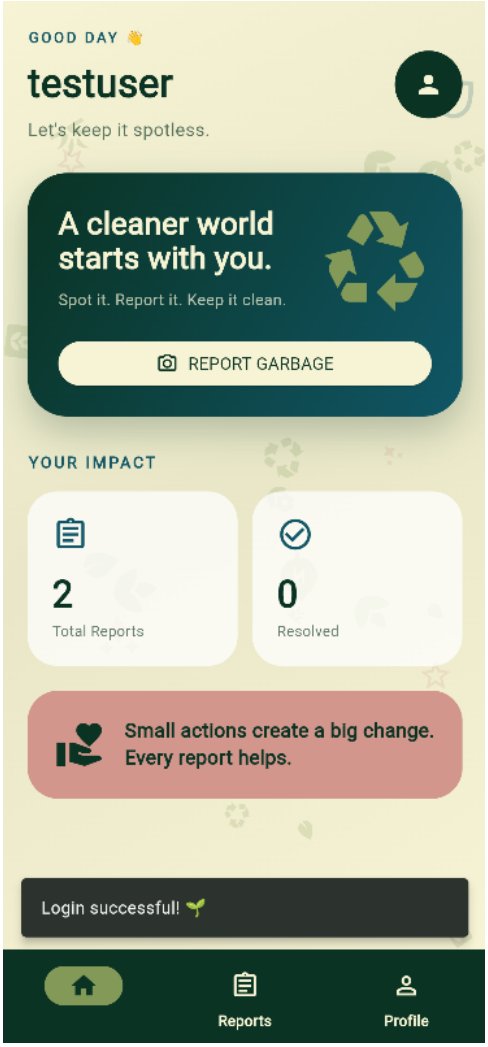


Mobile View

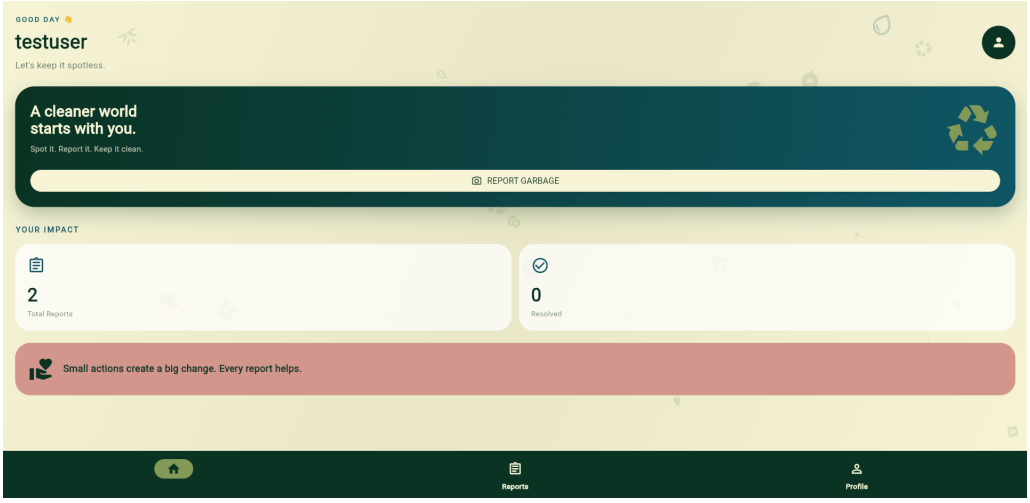


Desktop View

Home Screen

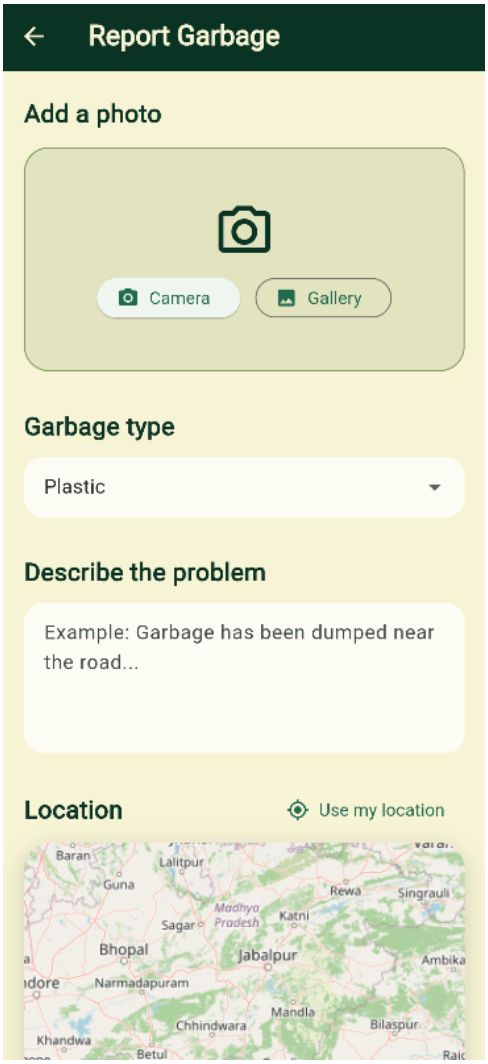


Mobile View

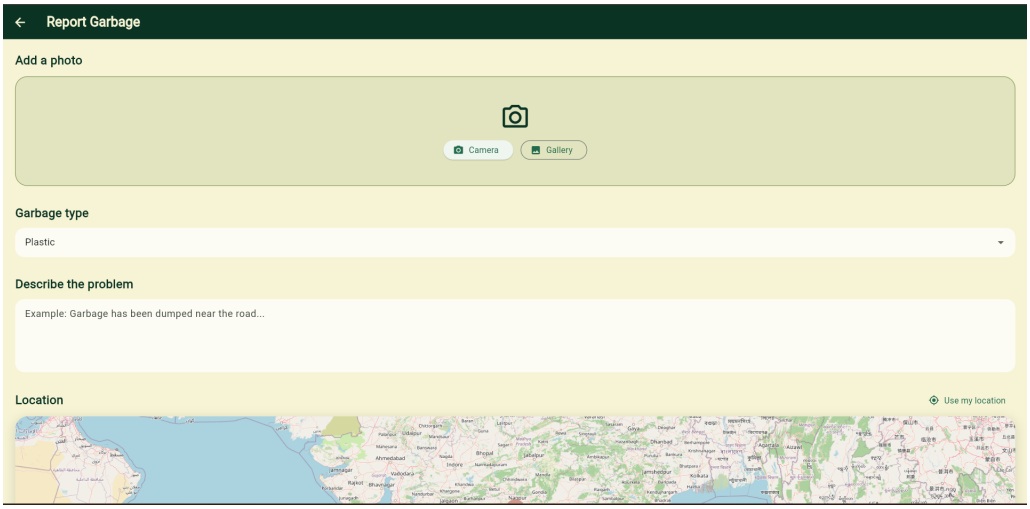


Desktop View

Report Screen

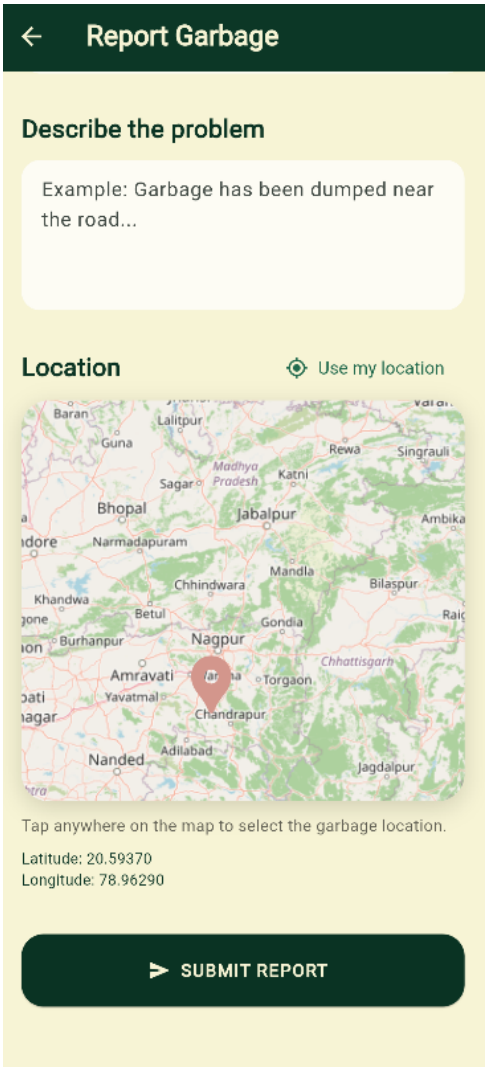


Mobile View

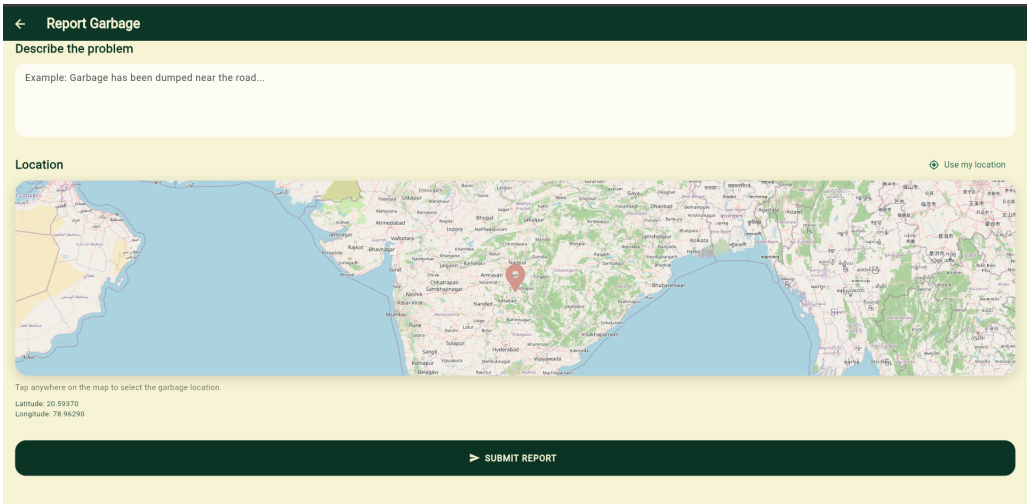


Desktop View

Report Screen

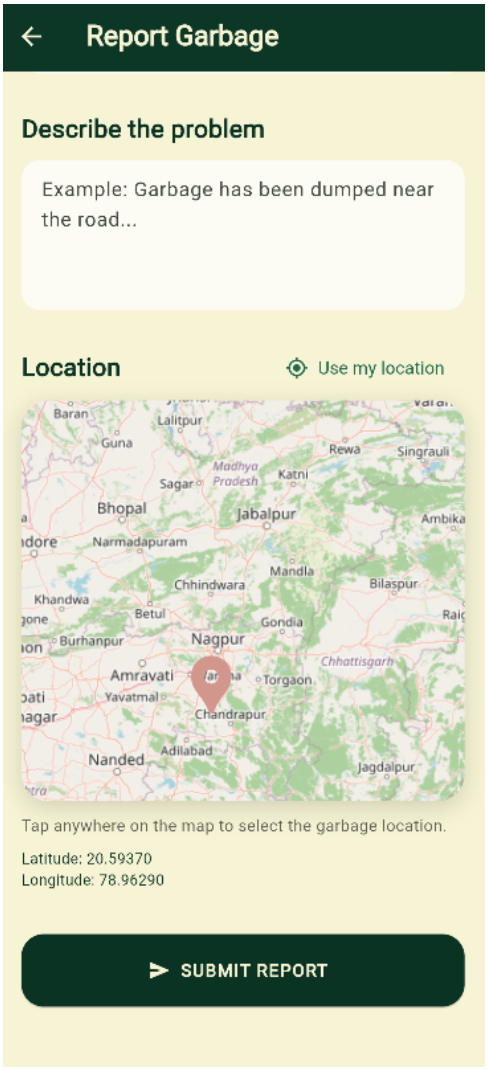


Mobile View

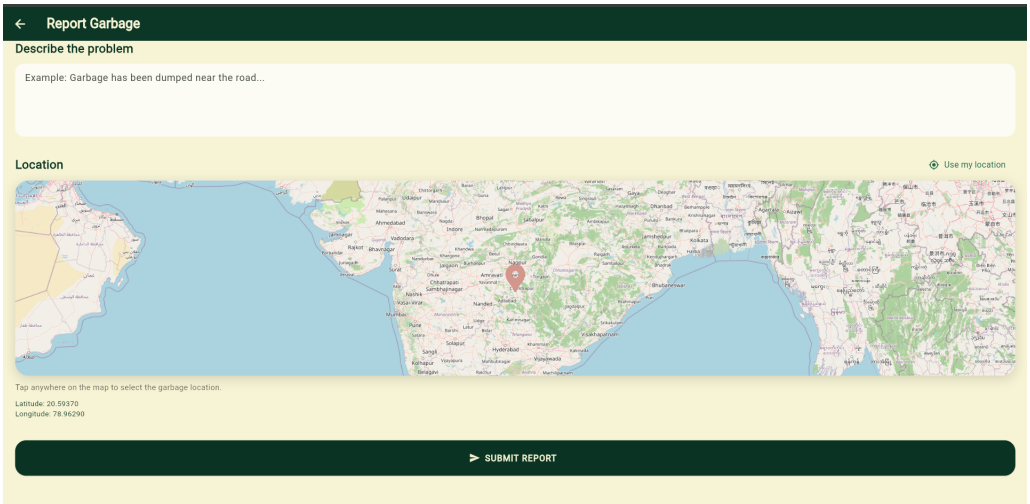


Desktop View

Reports Screen

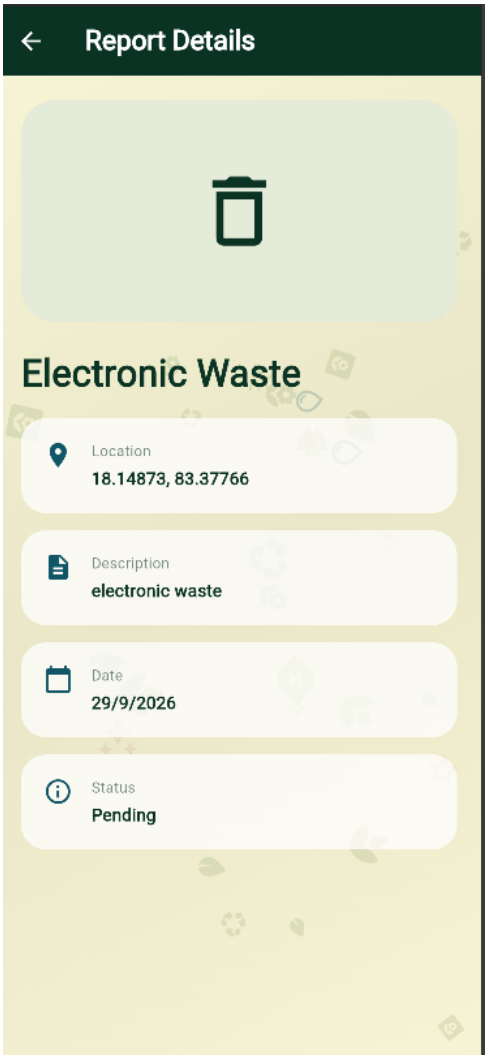


Mobile View

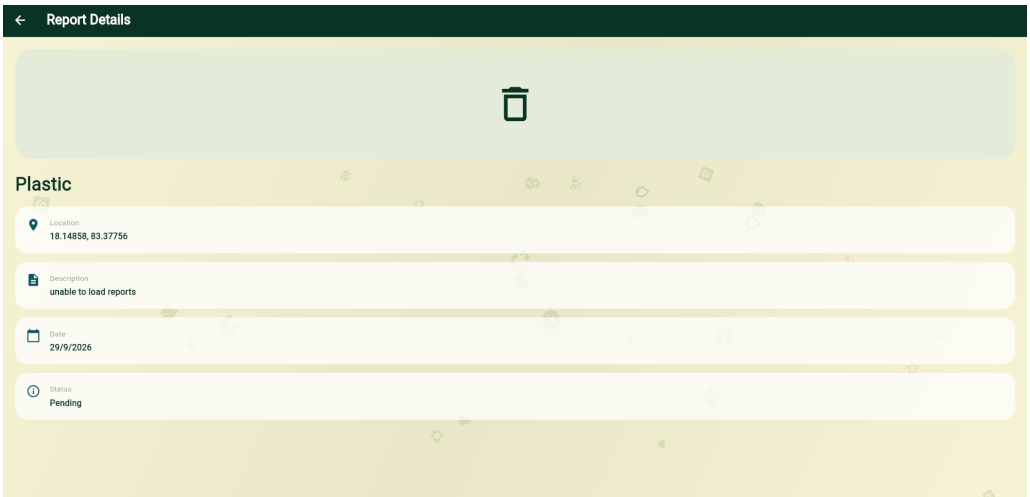


Desktop View

A Single Report Screen

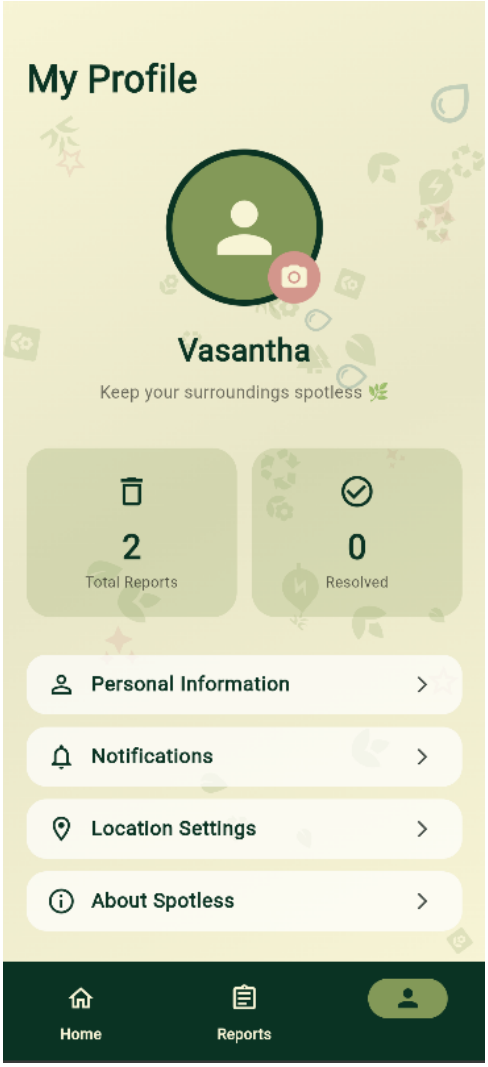


Mobile View

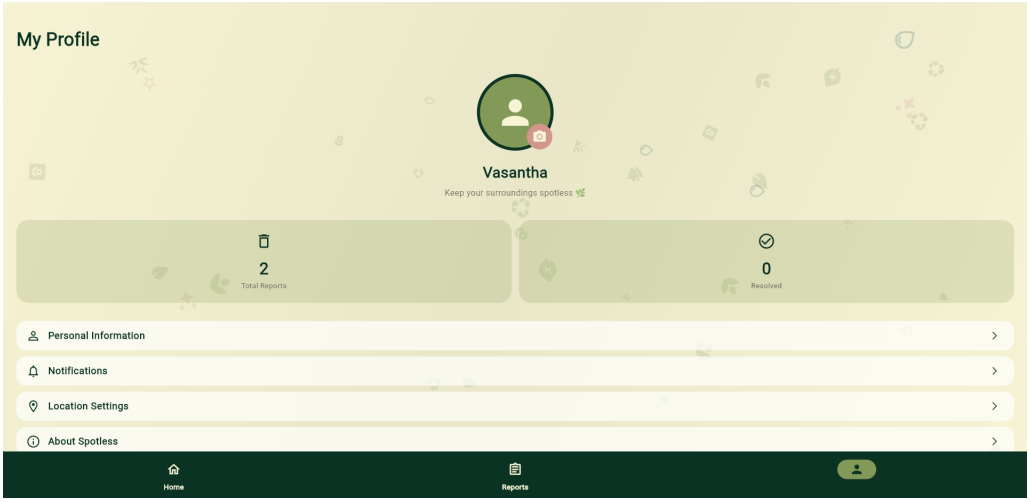


Desktop View

Profile Screen

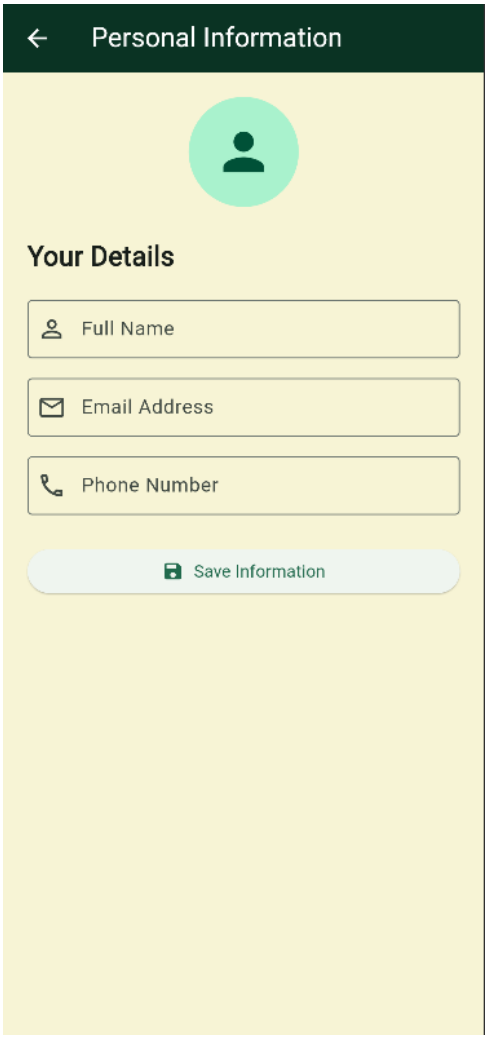


Mobile View

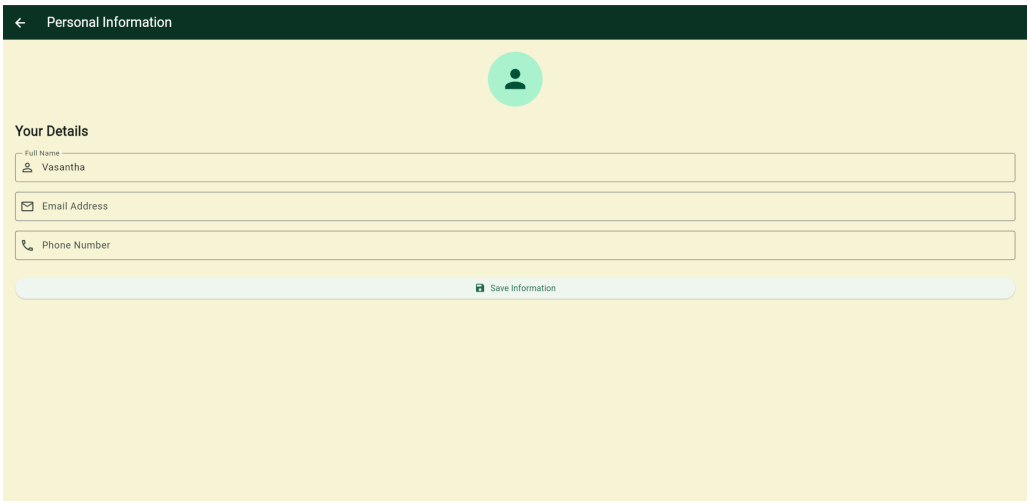


Desktop View

Personal Details Screen

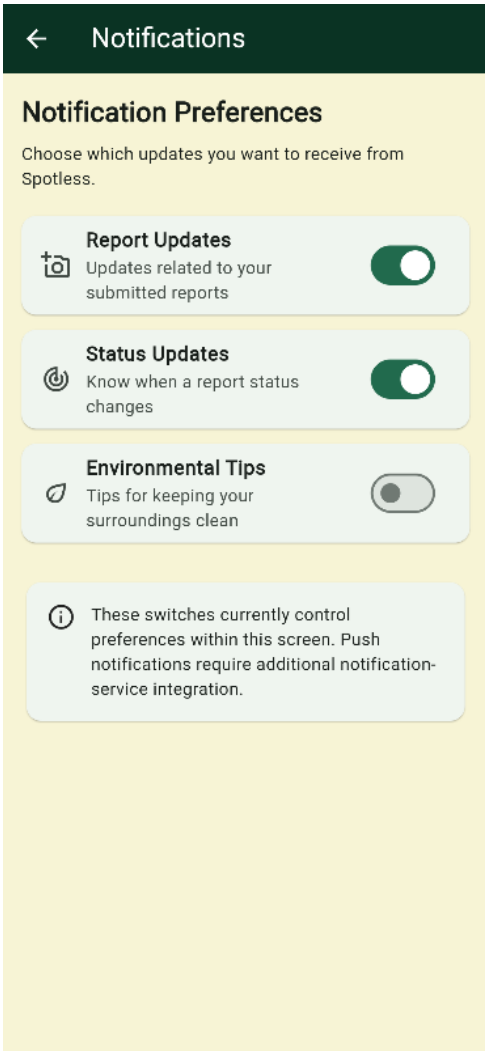


Mobile View

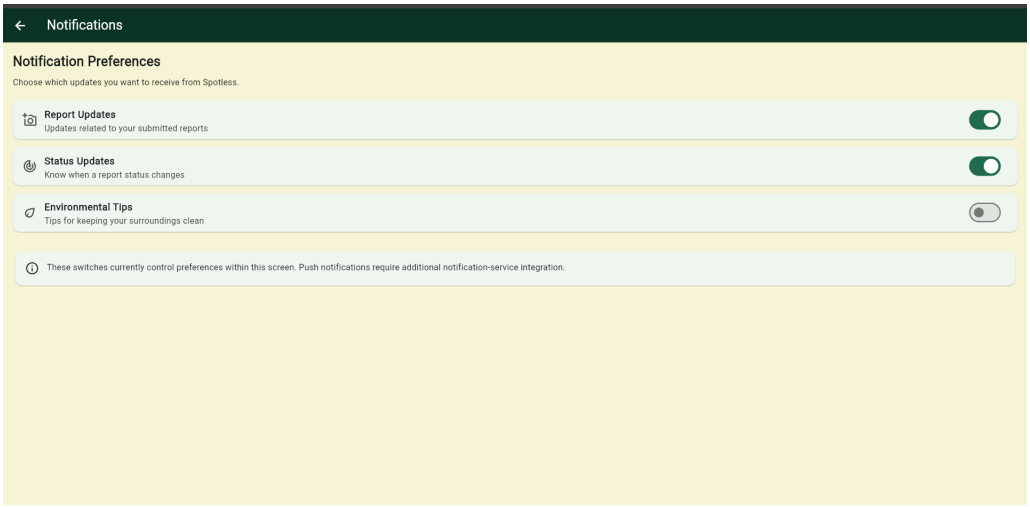


Desktop View

Notifications Screen

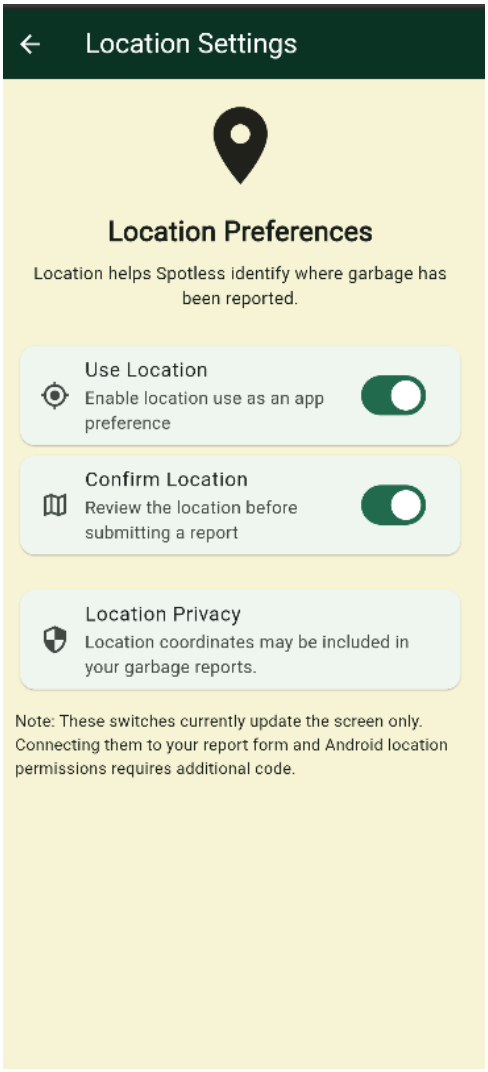


Mobile View

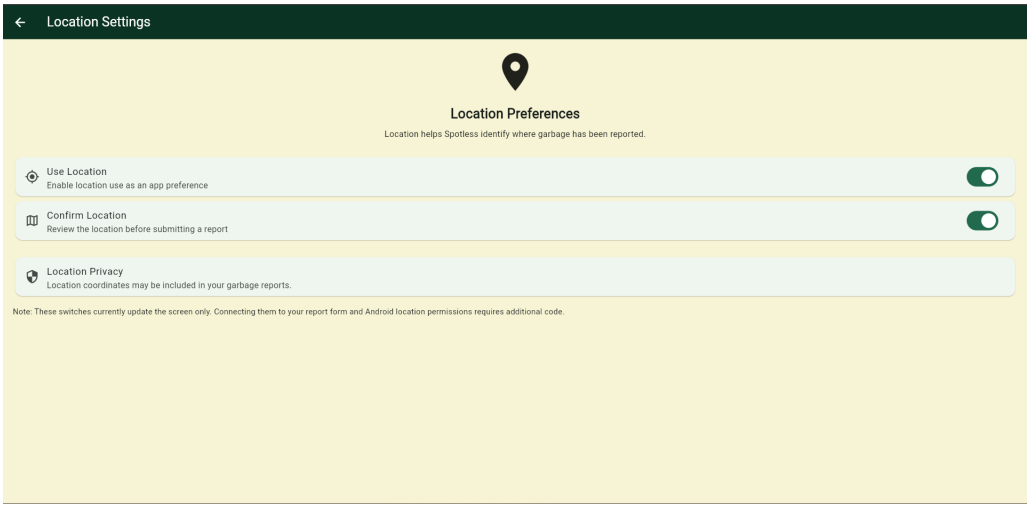


Desktop View

Location Settings Screen

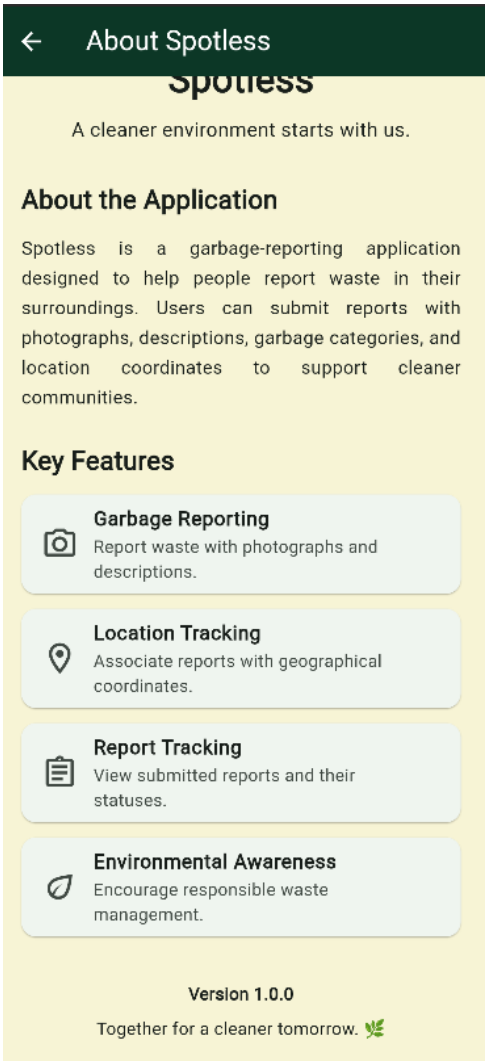


Mobile View

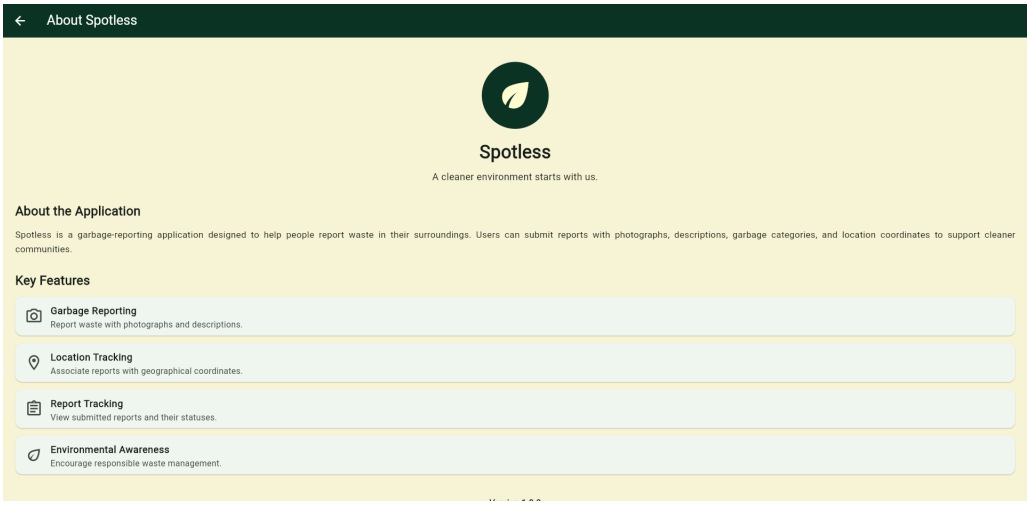


Desktop View

About Spotless Screen

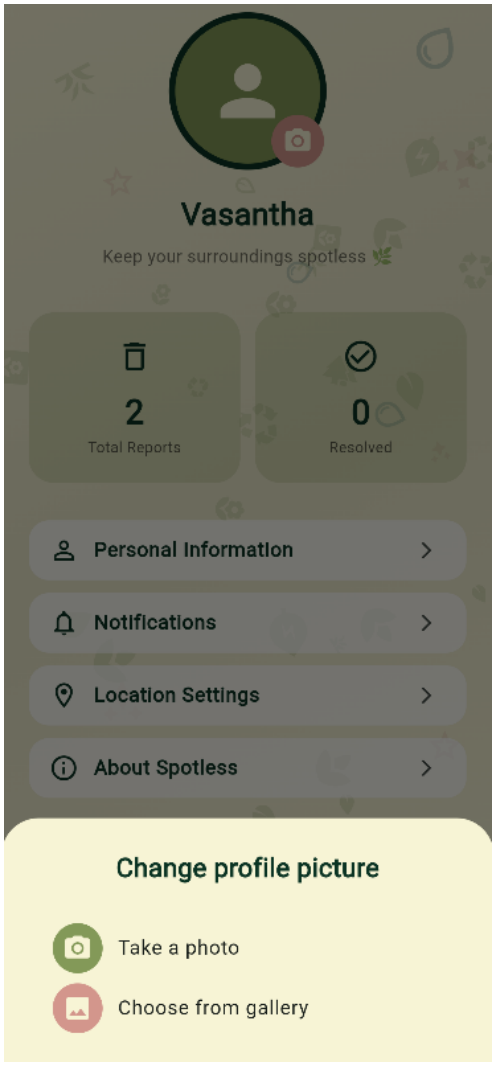


Mobile View

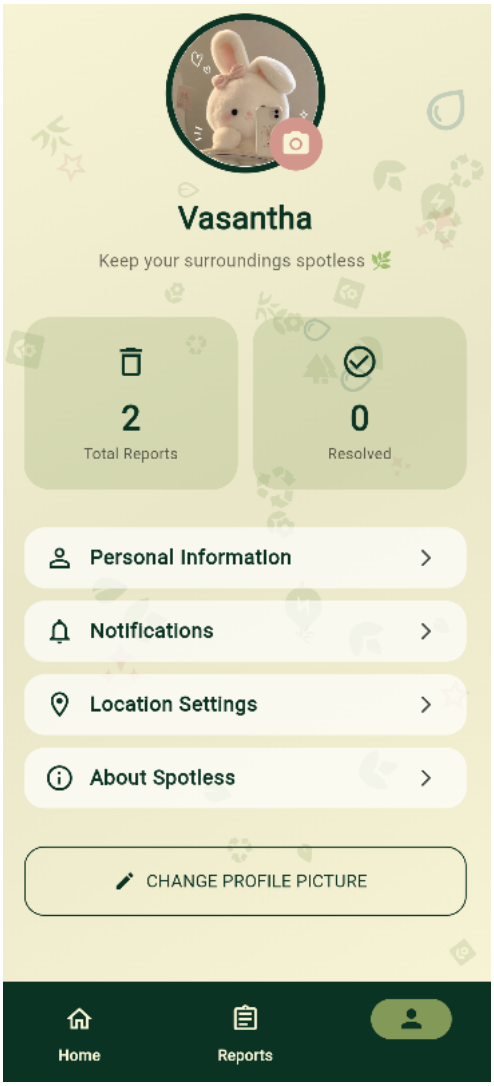


Desktop View

DP Editing Screen



Mobile View



Mobile View

11. Advantages

- Provides a structured way to report garbage in public places.
- Combines a category, description, photograph, and location in one report.
- Uses Flutter for a consistent cross-platform UI.
- Uses a shared MySQL database for accounts and report records.
- Makes report status and previously submitted reports easier to review.

12. Limitations

- The phone must be able to reach the backend for server-based login and report operations.
- The backend must be running and correctly configured while using a locally hosted server.
- The image is not necessarily uploaded as a file unless image-upload handling is implemented by the backend.
- Production deployment requires secure password hashing, protected endpoints, input validation, and HTTPS.

13. Future Enhancements

- Deploy the backend to a reliable hosting service so the laptop does not need to remain on.
- Upload report photographs to managed storage and save their URLs.
- Add notifications when report status changes.
- Add report filtering by category, date, and status.
- Provide an administrative interface for reviewing and resolving reports.
- Add secure authentication, stronger validation, and automated tests.

14. Conclusion

Spotless demonstrates how Flutter and Dart can be used to create a garbage-reporting application with account access, report submission, photographs, geographic coordinates, and report history. The Express.js backend and MySQL database provide shared storage for user accounts and reports. The project can be extended with hosted services, notifications, stronger security, and additional reporting tools.